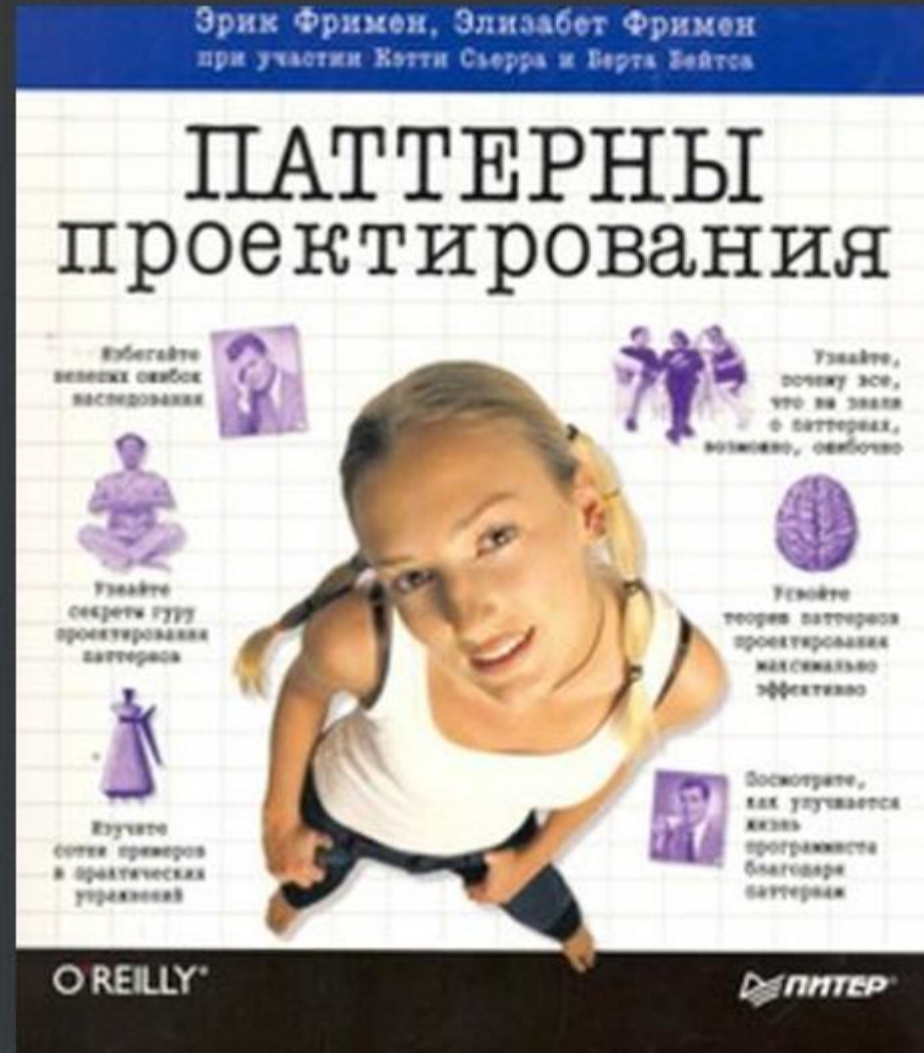
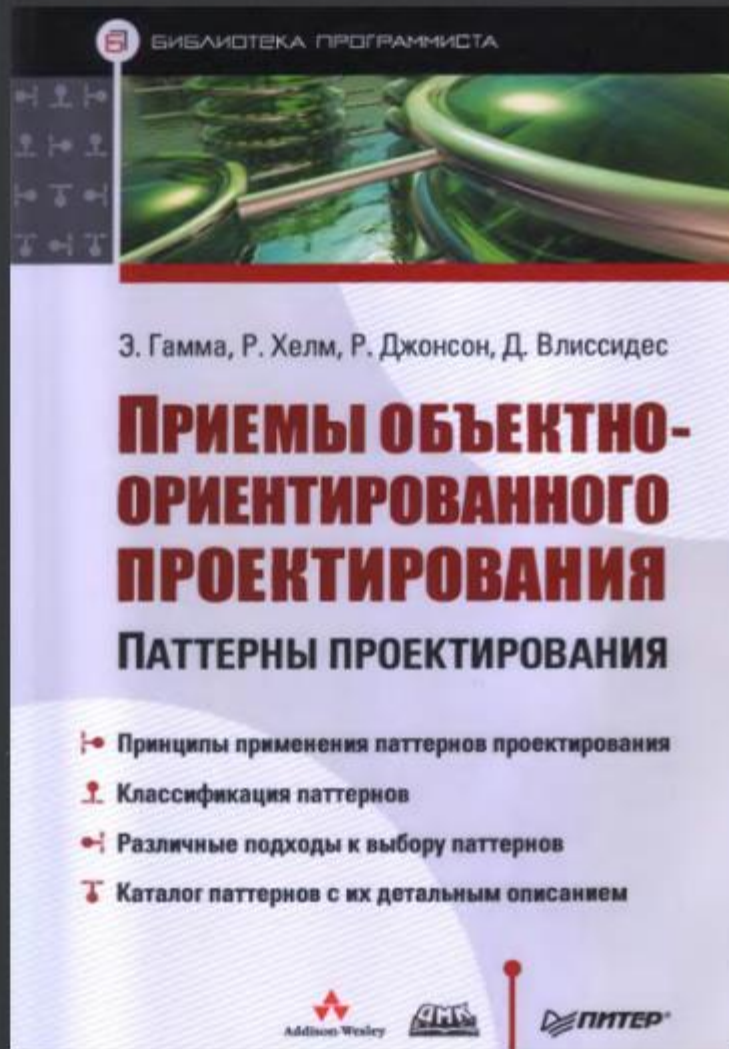


Тема 13. Паттерны проектирования

- 1. Что такое Design Patterns?**
- 2. Классификация паттернов**
- 3. Общая структура паттерна**
- 4. Первый паттерн: Singleton**

Полезная литература



Что такое “паттерны проектирования”?

«Любой паттерн описывает задачу, которая снова и снова возникает в нашей работе, а также принцип её решения, причём таким образом, что это решение можно потом использовать миллион раз, ничего не изобретая заново»

К. Александер (архитектор)

Разные контексты

- ☐ объектно-ориентированное проектирование (GoF)
- ☐ архитектурные шаблоны для бизнес-приложений
- ☐ шаблоны для систем реального времени
- ☐ шаблоны написания Вариантов Исползования
- ☐ game programming patterns
- ☐ шаблоны реактивного программирования
- ☐ антипаттерны

Паттерны объектно-ориентированного проектирования

- ☐ **шаблоны “Банды четырех” (Gang of Four, GoF)**
- ☐ **описание взаимодействия объектов и классов, адаптированных для решения общей задачи проектирования в конкретном контексте**
 - ✓ **ООП**
 - ✓ **диаграммы классов / последовательности (состояний)**
 - ✓ **повторное использование**
 - ✓ **низкая связанность**

Паттерны GoF

23 паттерна классифицируются

- ☐ **по назначению**
 - ✓ **порождающие паттерны**
 - ✓ **структурные паттерны**
 - ✓ **паттерны поведения**

- ☐ **по уровню применения**
 - ✓ **уровня классов**
 - ✓ **уровня объектов**

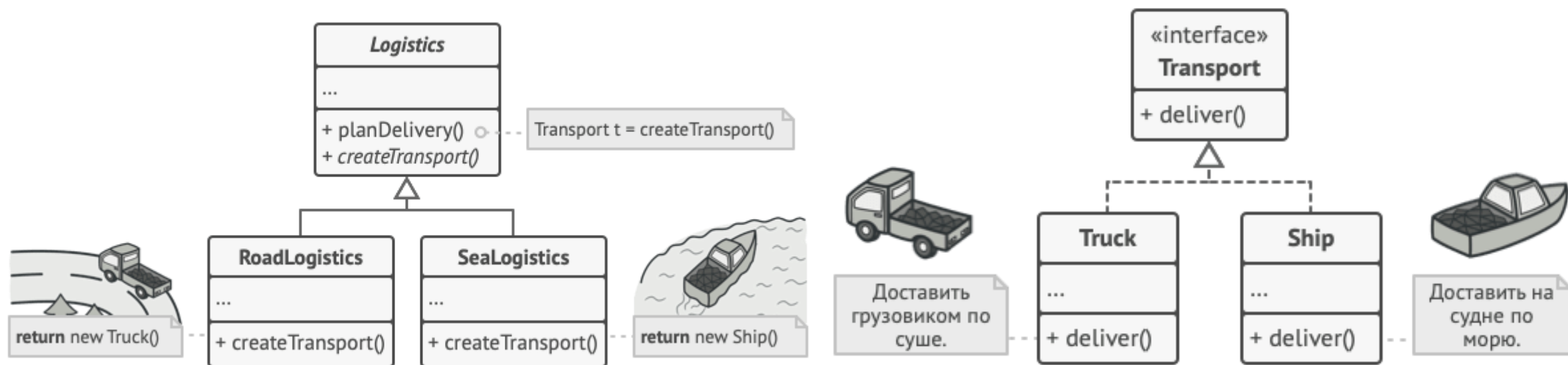
Порождающие паттерны проектирования

- **Фабричный метод** (Factory Method)
- **Абстрактная фабрика** (Abstract Factory)
- **Строитель** (Builder)
- **Прототип** (Prototype)
- **Одиночка** (Singleton)

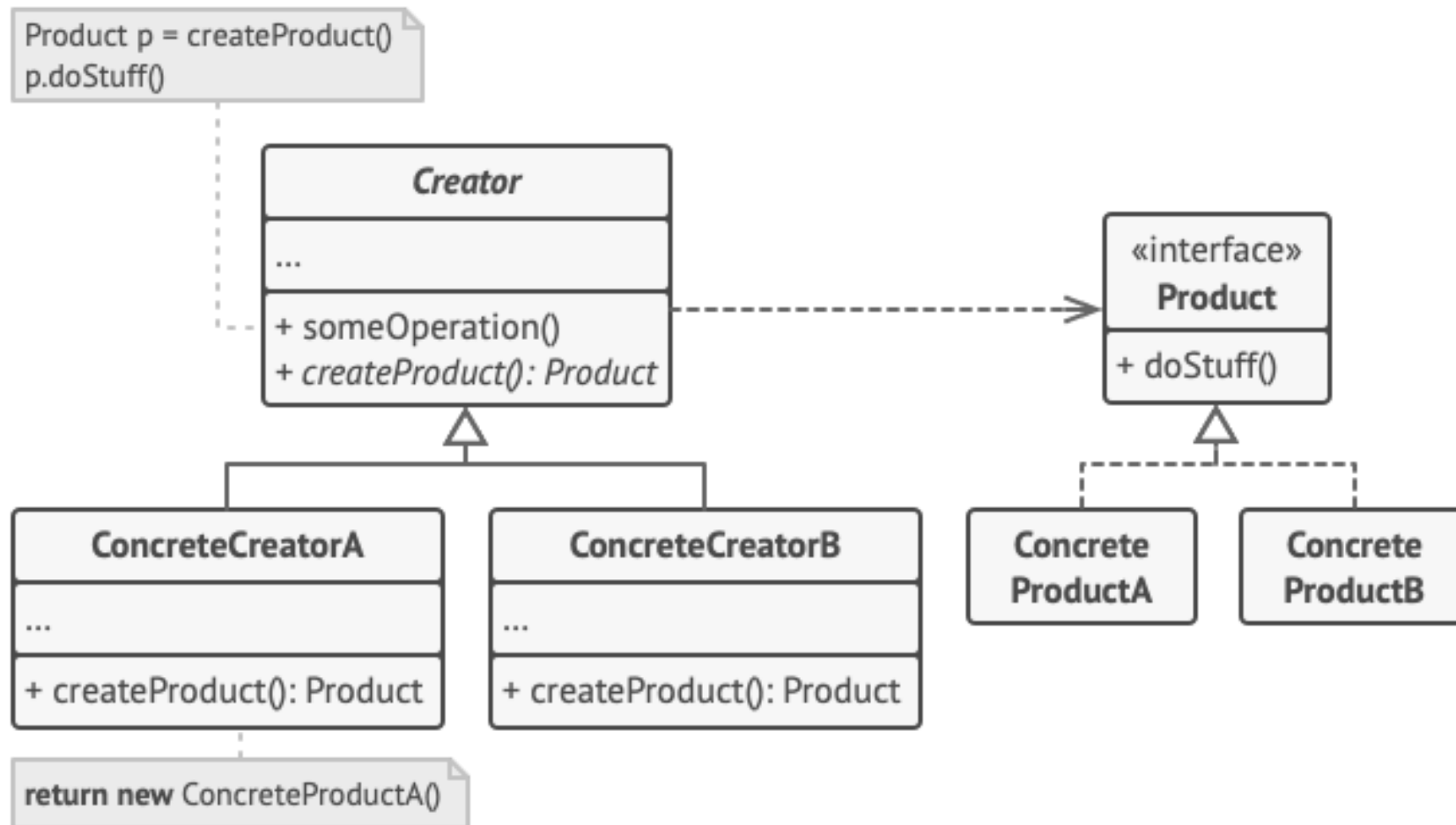
Фабричный метод (Factory Method)



Фабричный метод (Factory Method)



Фабричный метод (Factory Method)



- **Продукт** определяет общий интерфейс объектов, которые может произвести создатель и его подклассы.
- **Конкретные продукты** содержат код различных продуктов. Продукты будут отличаться реализацией, но интерфейс у них будет общий.
- **Создатель** объявляет фабричный метод, который должен возвращать новые объекты продуктов. Важно, чтобы тип результата совпадал с общим интерфейсом продуктов.
- **Конкретные создатели** по-своему реализуют фабричный метод, производя те или иные конкретные продукты.

Преимущества и недостатки

- Избавляет класс от привязки к конкретным классам продуктов.
- Выделяет код производства продуктов в одно место, упрощая поддержку кода.
- Упрощает добавление новых продуктов в программу.
- Реализует *принцип открытости/закрытости*.
- Может привести к созданию больших **параллельных иерархий классов**, так как для каждого класса продукта надо создать свой подкласс создателя.

Абстрактная фабрика (Abstract Factory)

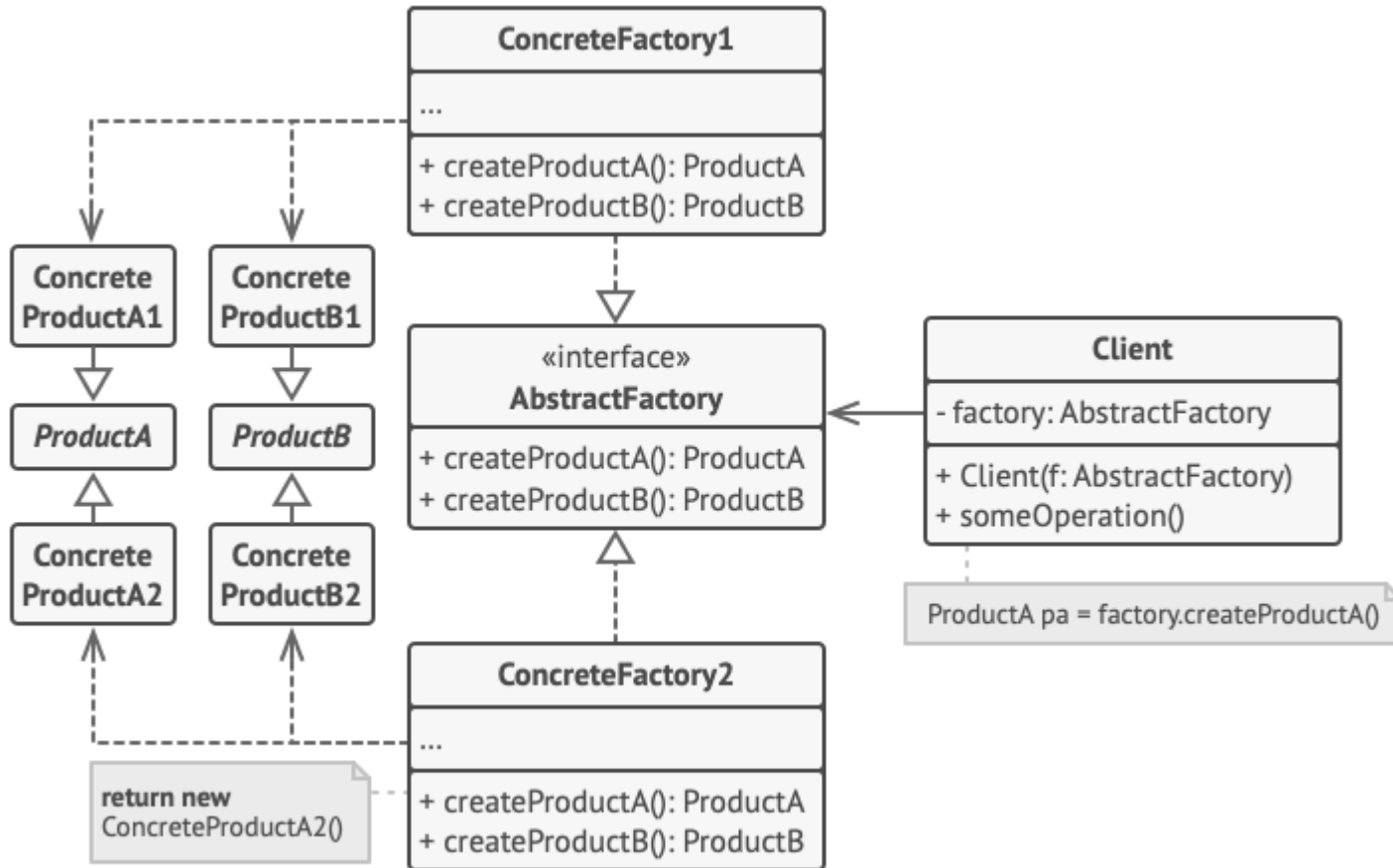


Абстрактная фабрика (Abstract Factory)

	Кресло	Диван	Столик
Ар-деко			
Виктори-анский			
Модерн			

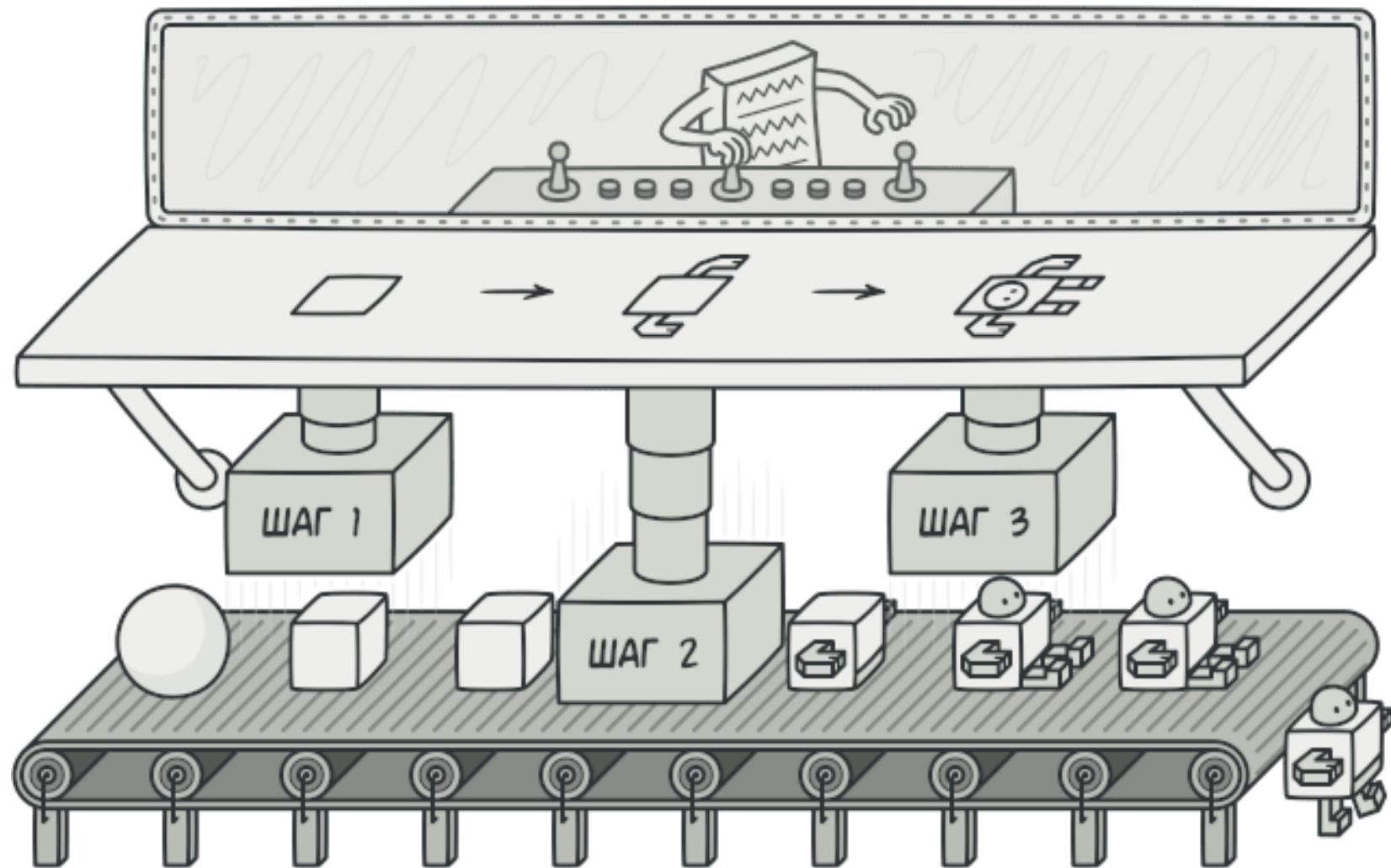


Абстрактная фабрика (Abstract Factory)

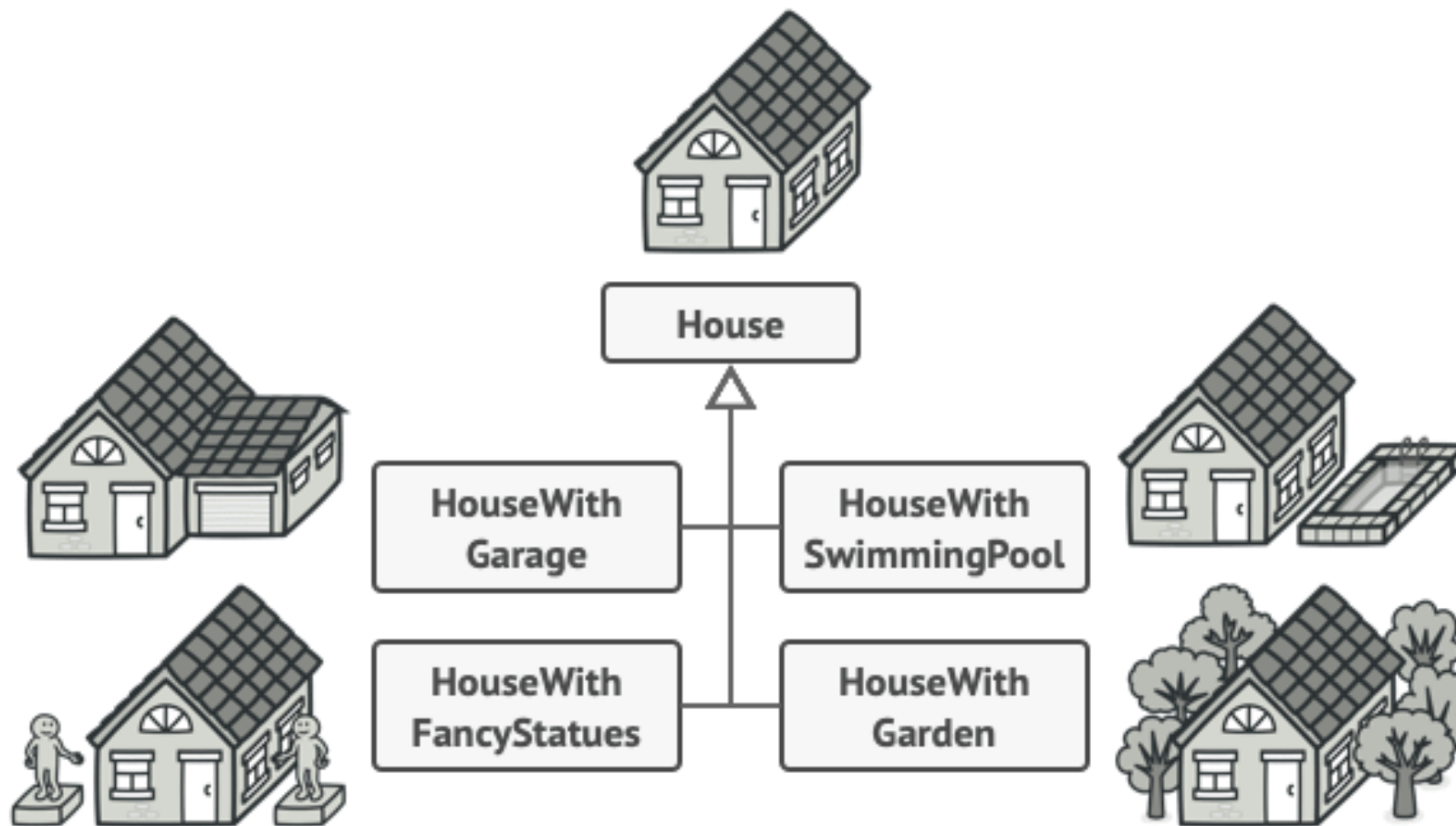


- **Абстрактные продукты** объявляют интерфейсы продуктов, которые связаны друг с другом по смыслу, но выполняют разные функции.
- **Конкретные продукты** — большой набор классов, которые относятся к различным абстрактным продуктам (кресло/столик), но имеют одни и те же вариации (Викторианский/Модерн).
- **Абстрактная фабрика** объявляет методы создания различных абстрактных продуктов (кресло/столик).
- **Конкретные фабрики** относятся каждая к своей вариации продуктов (Викторианский/Модерн) и реализуют методы абстрактной фабрики, позволяя создавать все продукты определённой вариации.
- Несмотря на то, что конкретные фабрики порождают конкретные продукты, сигнатуры их методов должны возвращать соответствующие абстрактные продукты. Это позволит клиентскому коду, использующему фабрику, не привязываться к конкретным классам продуктов. Клиент сможет работать с любыми вариациями продуктов через абстрактные интерфейсы.

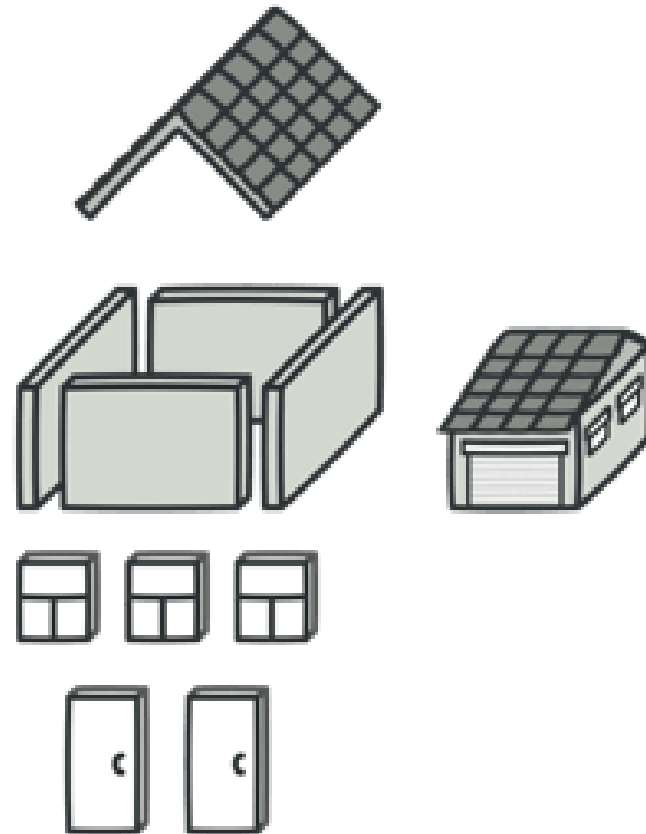
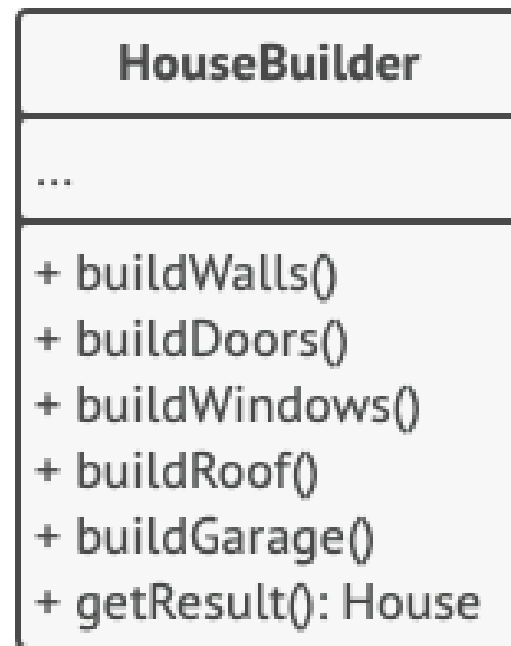
Строитель (Builder)



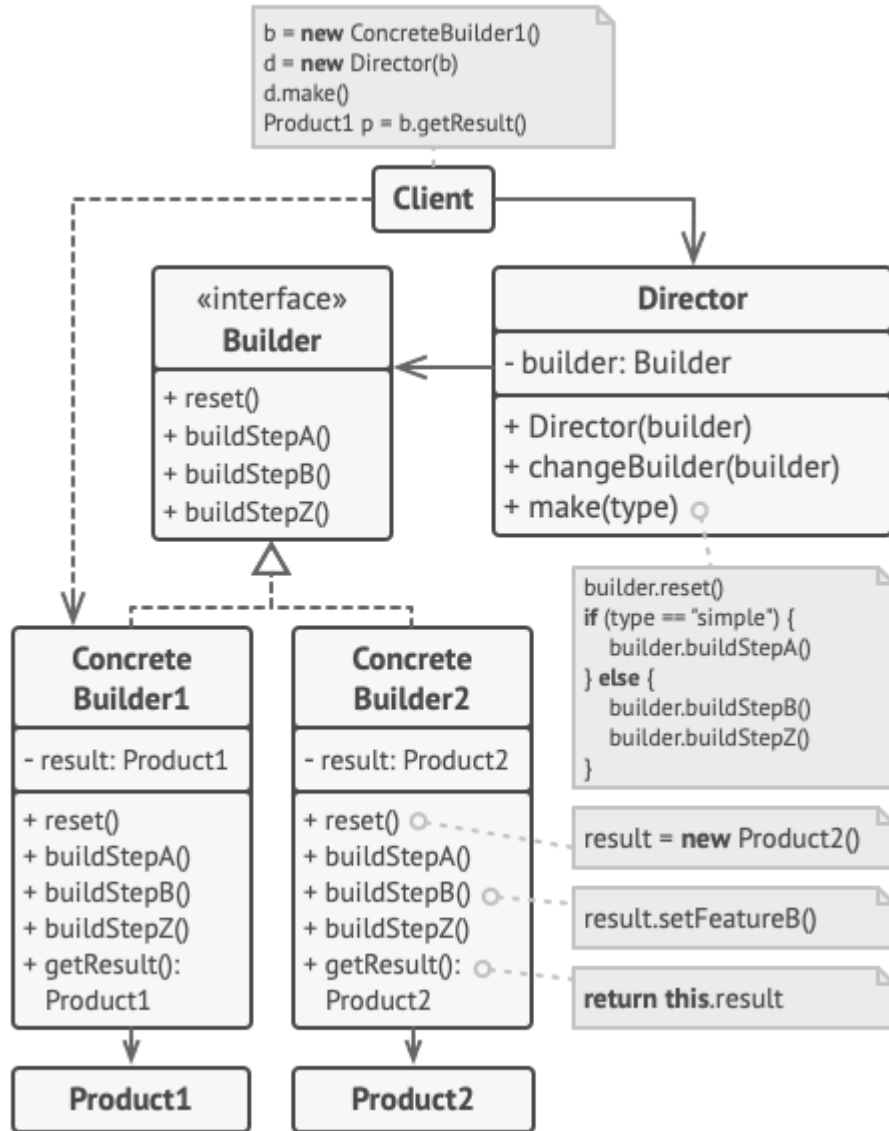
Проблема



Решение

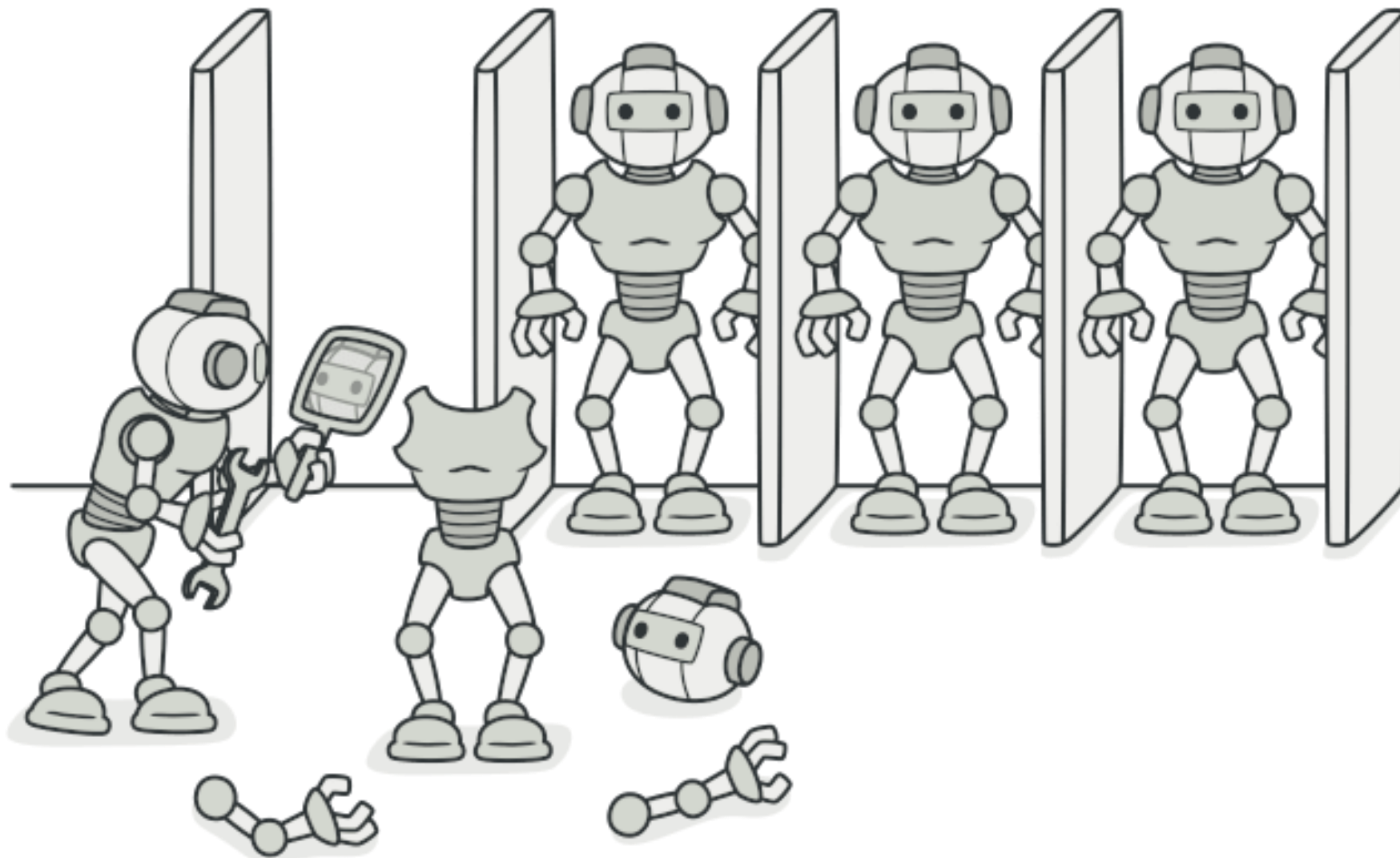


Структура

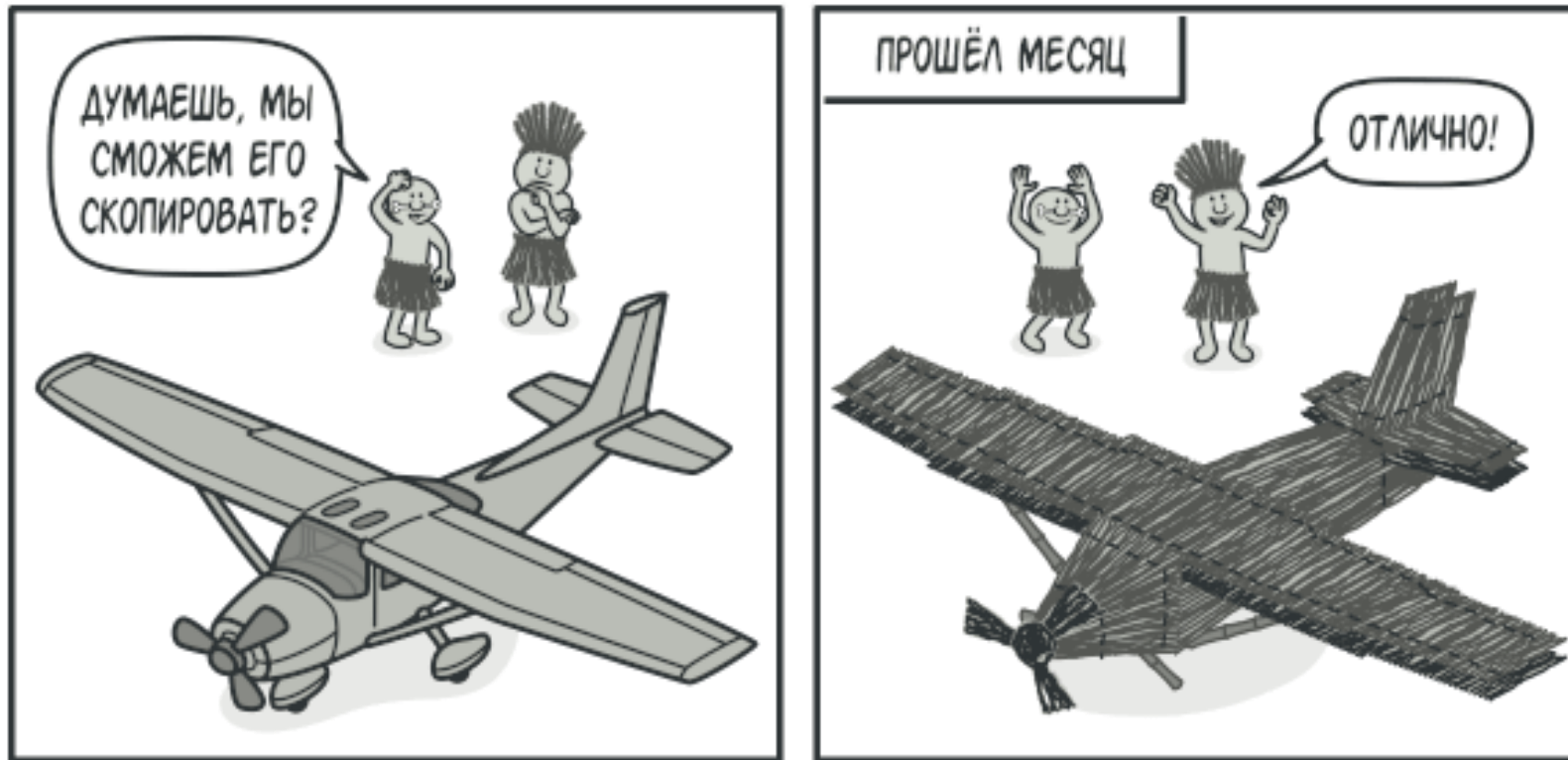


- **Интерфейс строителя** объявляет шаги конструирования продуктов, общие для всех видов строителей.
- **Конкретные строители** реализуют строительные шаги, каждый по-своему. Конкретные строители могут производить разнородные объекты, не имеющие общего интерфейса.
- **Продукт** — создаваемый объект. Продукты, сделанные разными строителями, не обязаны иметь общий интерфейс.
- **Директор** определяет порядок вызова строительных шагов для производства той или иной конфигурации продуктов.
- Обычно **Клиент** подаёт в конструктор директора уже готовый объект-строитель, и в дальнейшем данный директор использует только его. Но возможен и другой вариант, когда клиент передаёт строителя через параметр строительного метода директора. В этом случае можно каждый раз применять разных строителей для производства различных представлений объектов.

Прототип (Prototype)

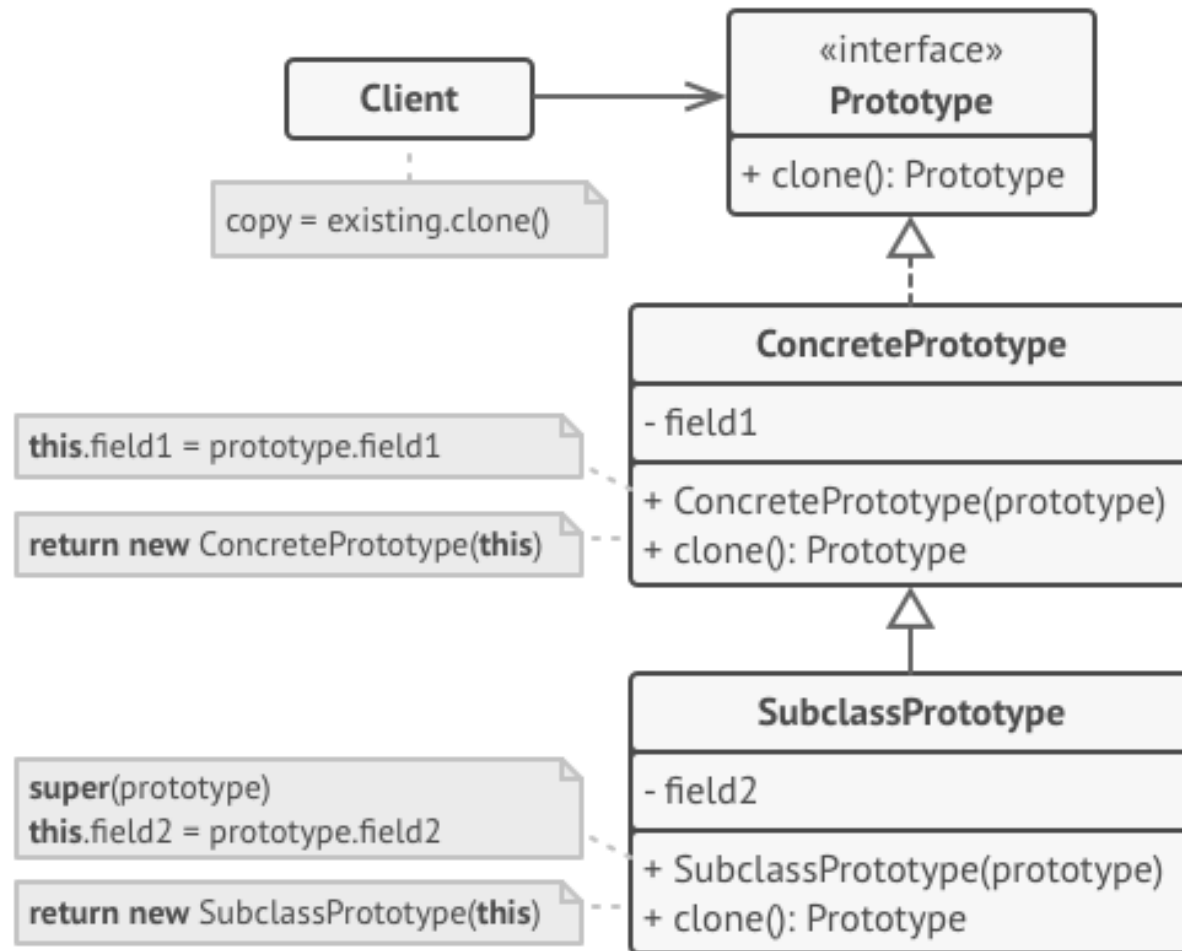


Проблема



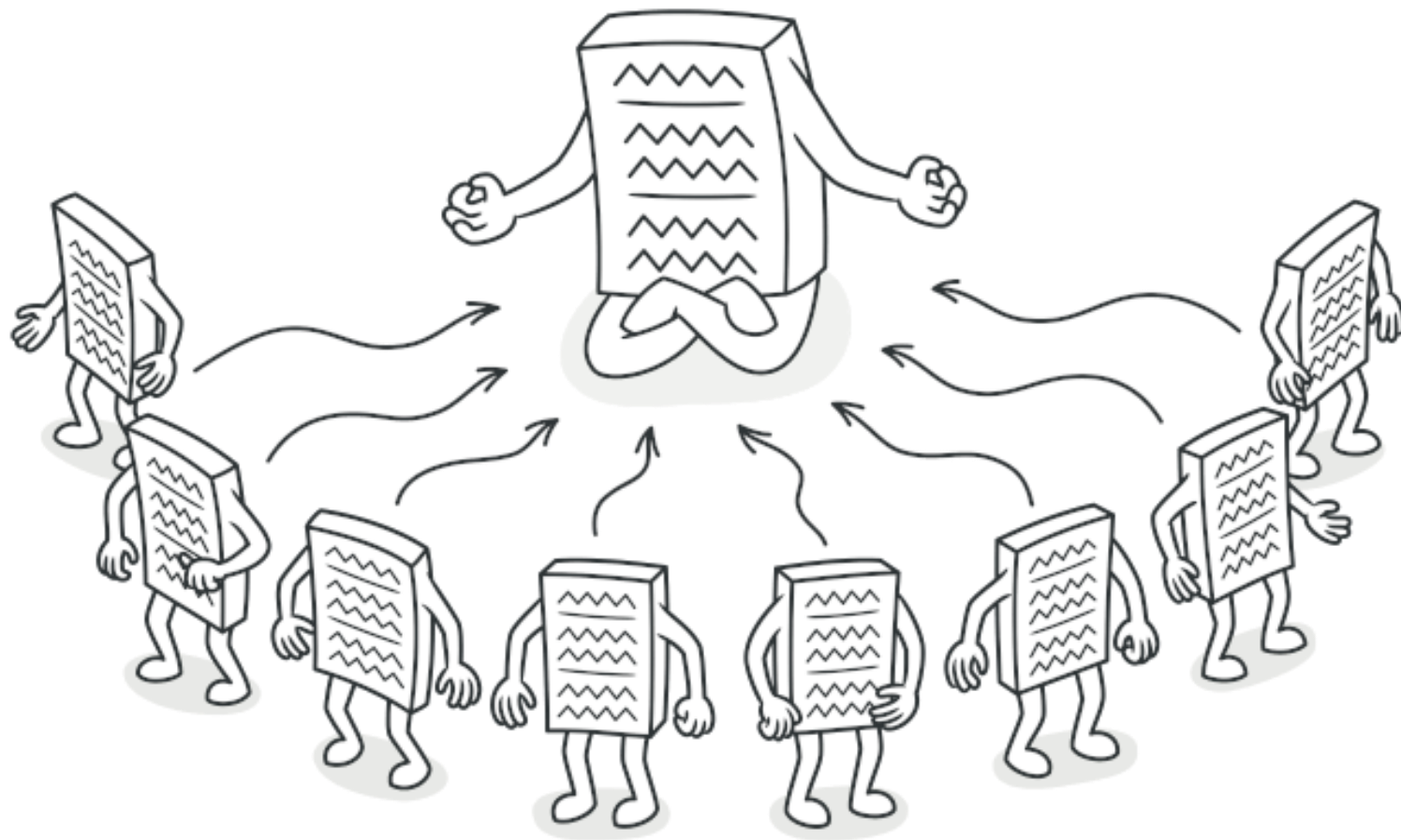
- Копирование «извне» не всегда возможно в реальности.

Структура

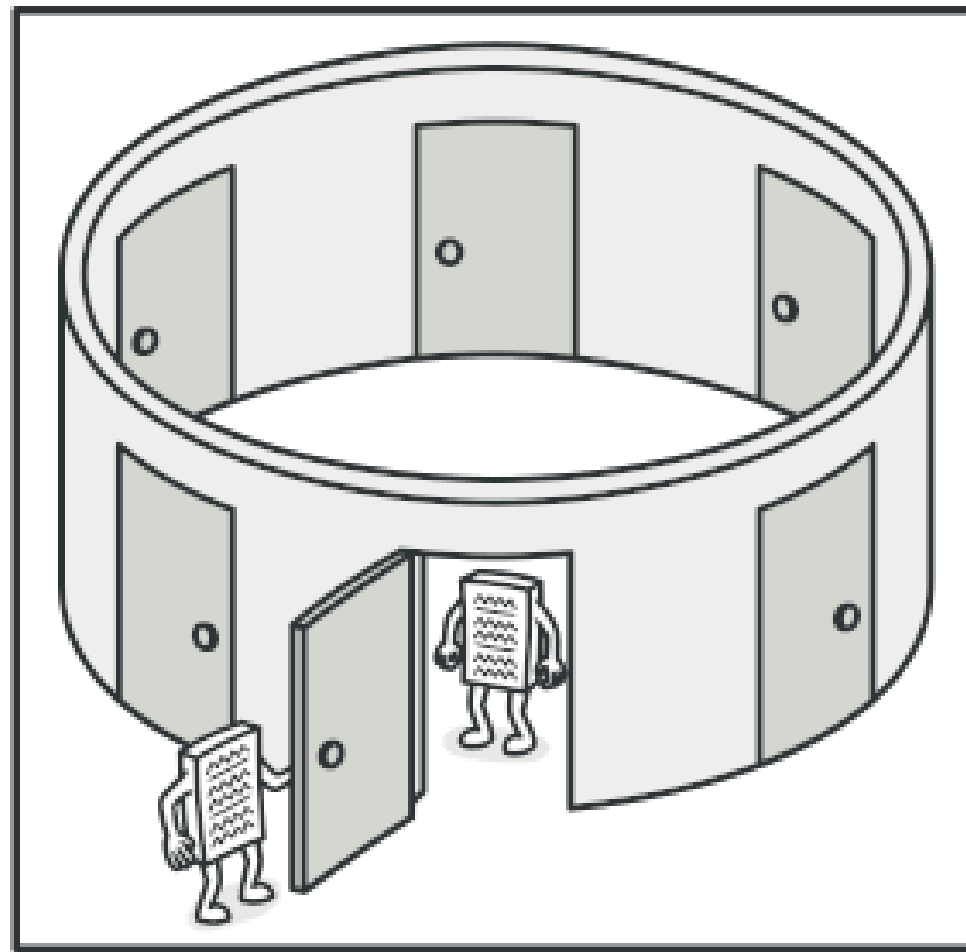


- Интерфейс прототипов описывает операции клонирования. В большинстве случаев — это единственный метод `clone`.
- **Конкретный прототип** реализует операцию клонирования самого себя. Помимо банального копирования значений всех полей, здесь могут быть спрятаны различные сложности, о которых не нужно знать клиенту. Например, клонирование связанных объектов, распутывание рекурсивных зависимостей и прочее.
- **Клиент** создаёт копию объекта, обращаясь к нему через общий интерфейс прототипов.

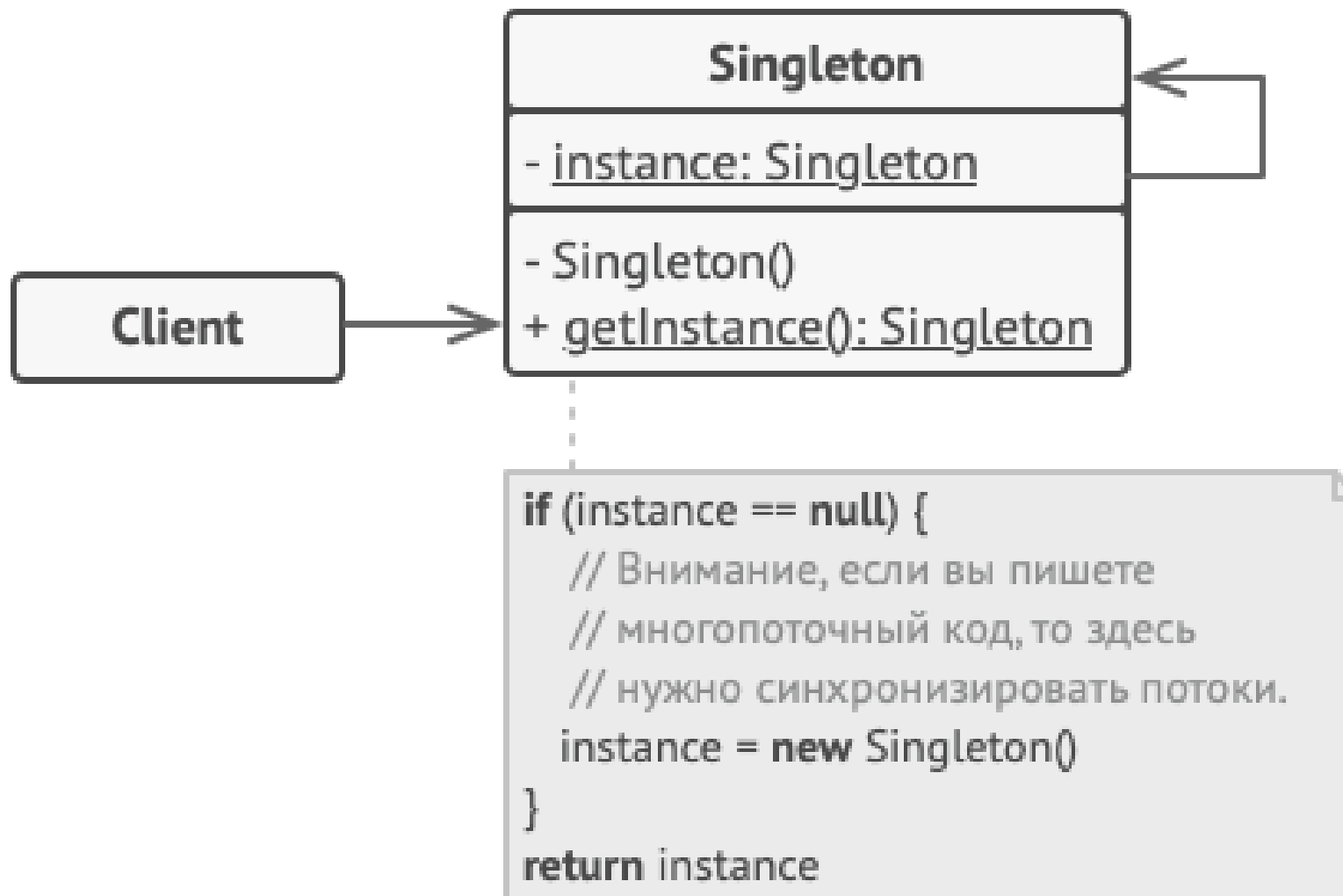
Одиночка (Singleton)



Проблема



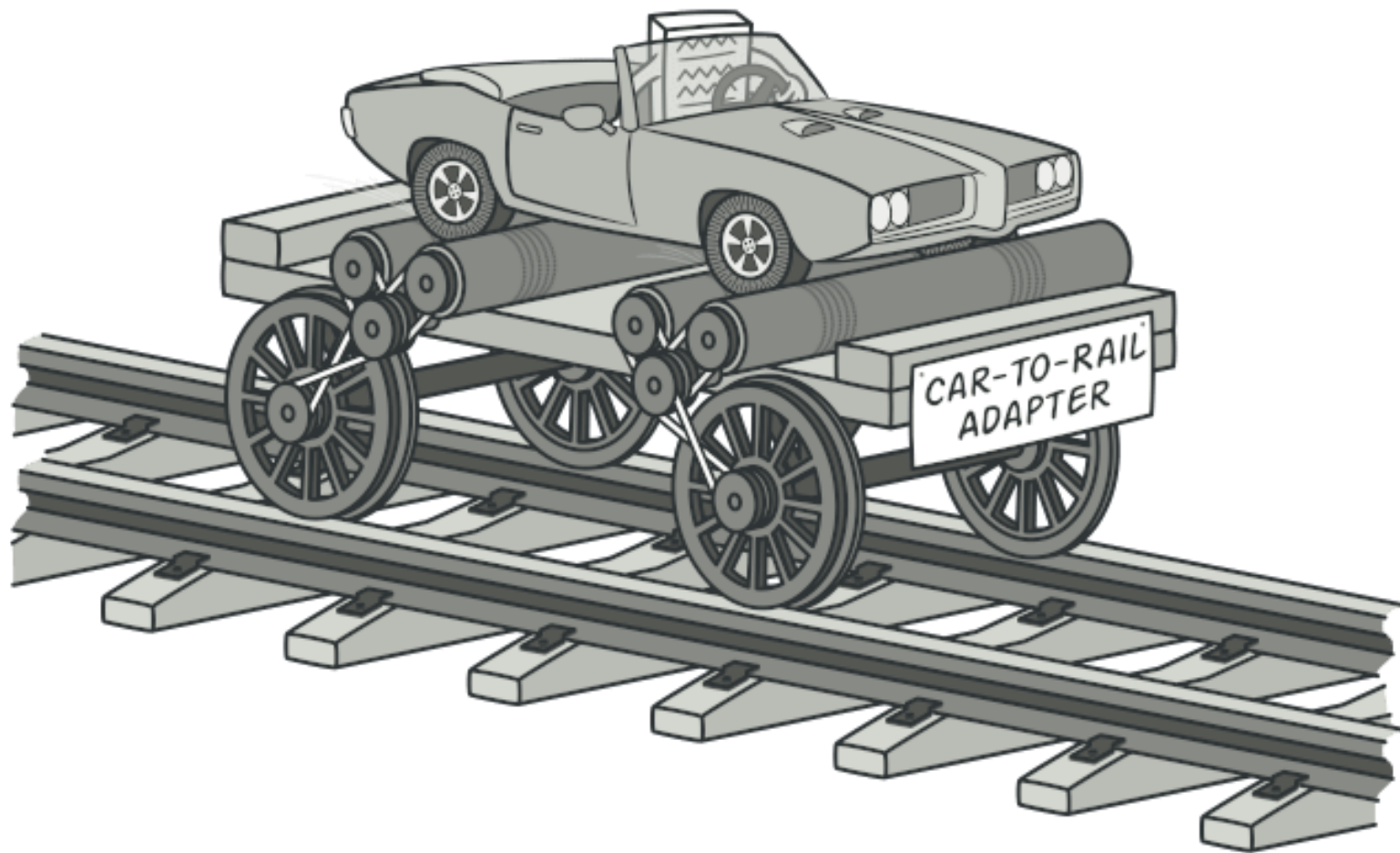
Решение



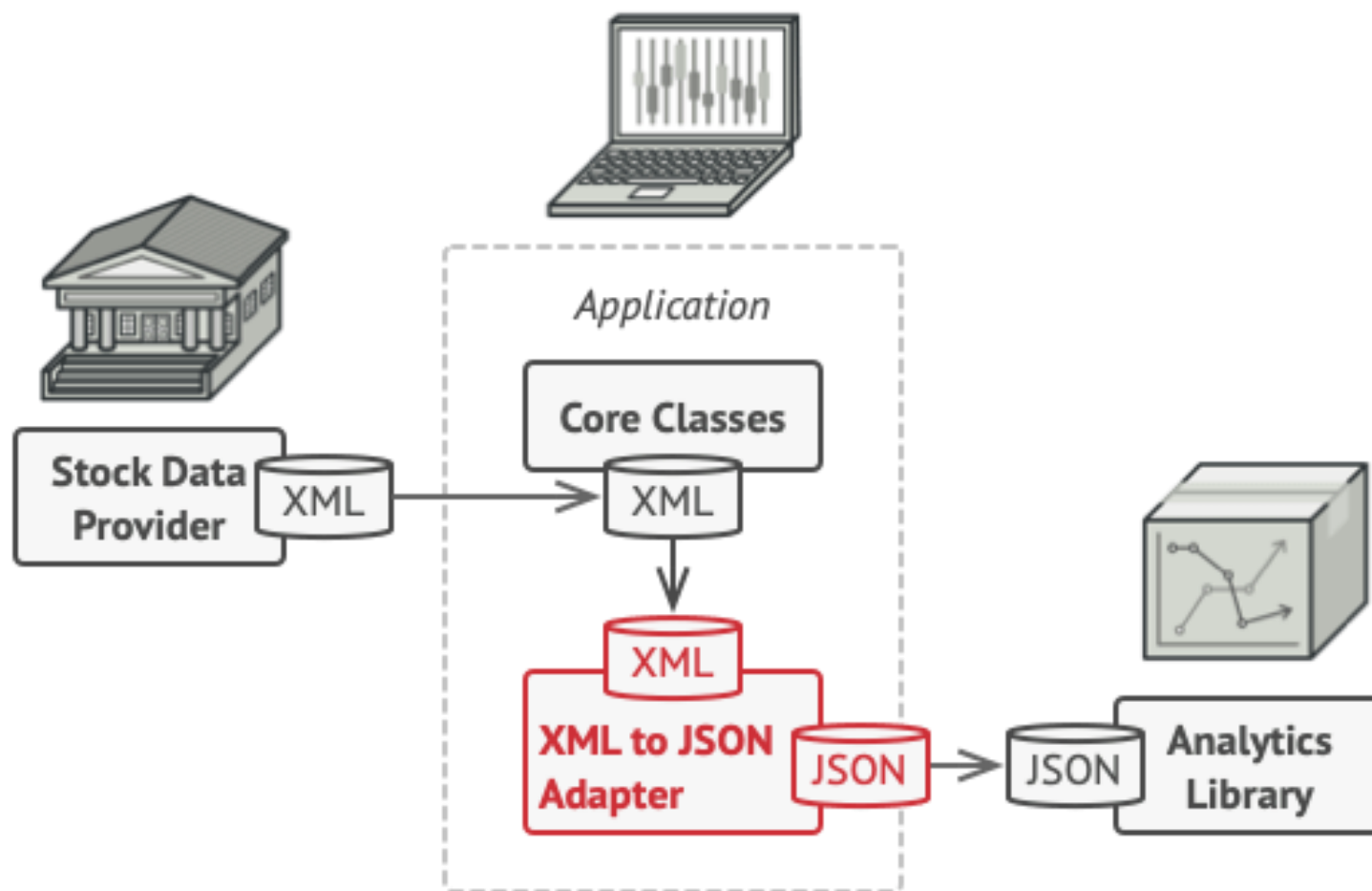
Структурные паттерны проектирования

- **Адаптер** (Adapter, Wrapper)
- **Мост** (Bridge)
- **Компоновщик** (Composite)
- **Декоратор** (Decorator, Wrapper)
- **Фасад** (Facade)
- **Легковес** (Flyweight)
- **Заместитель** (Proxy)

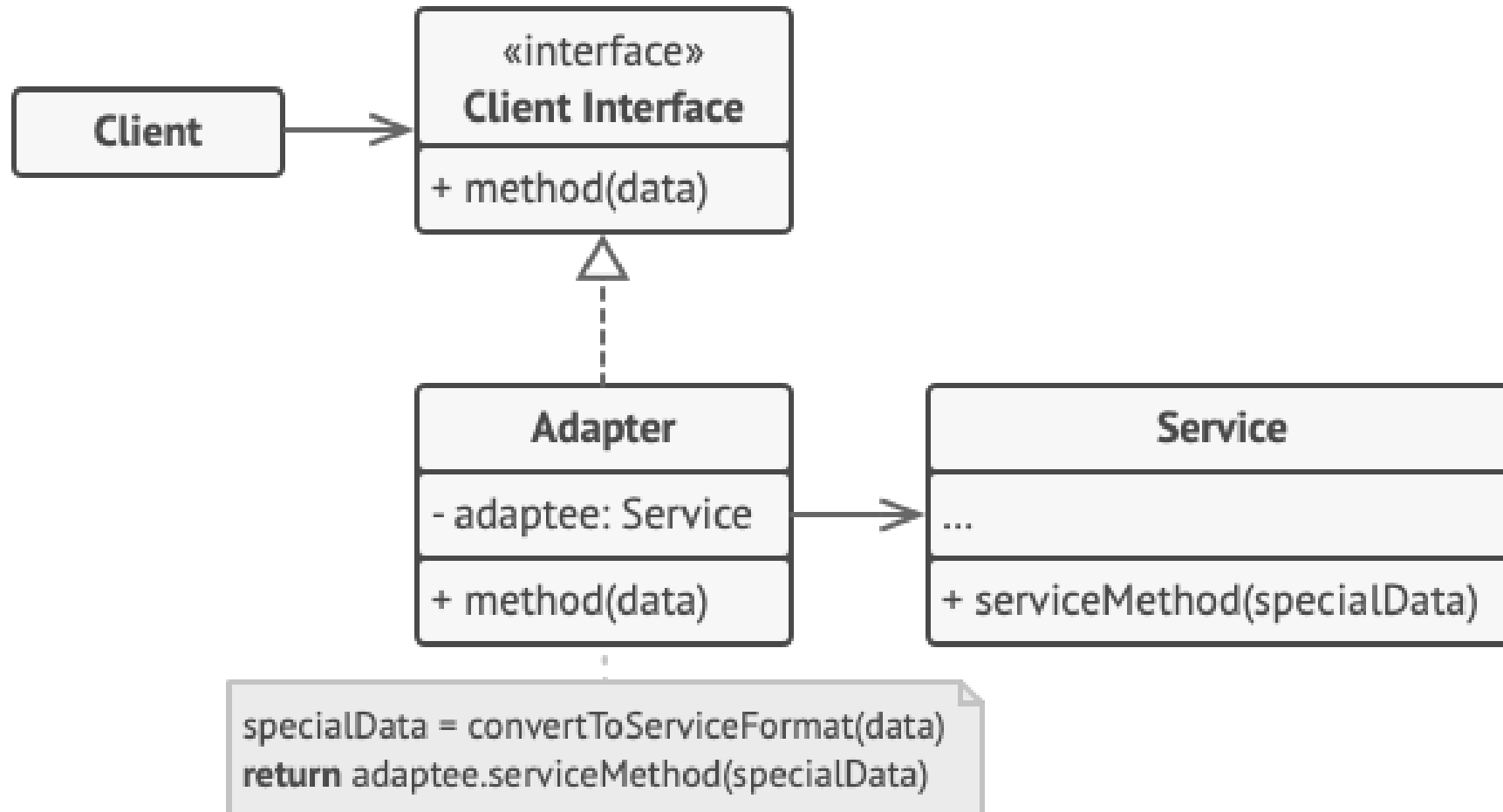
Адаптер (Adapter, Wrapper)



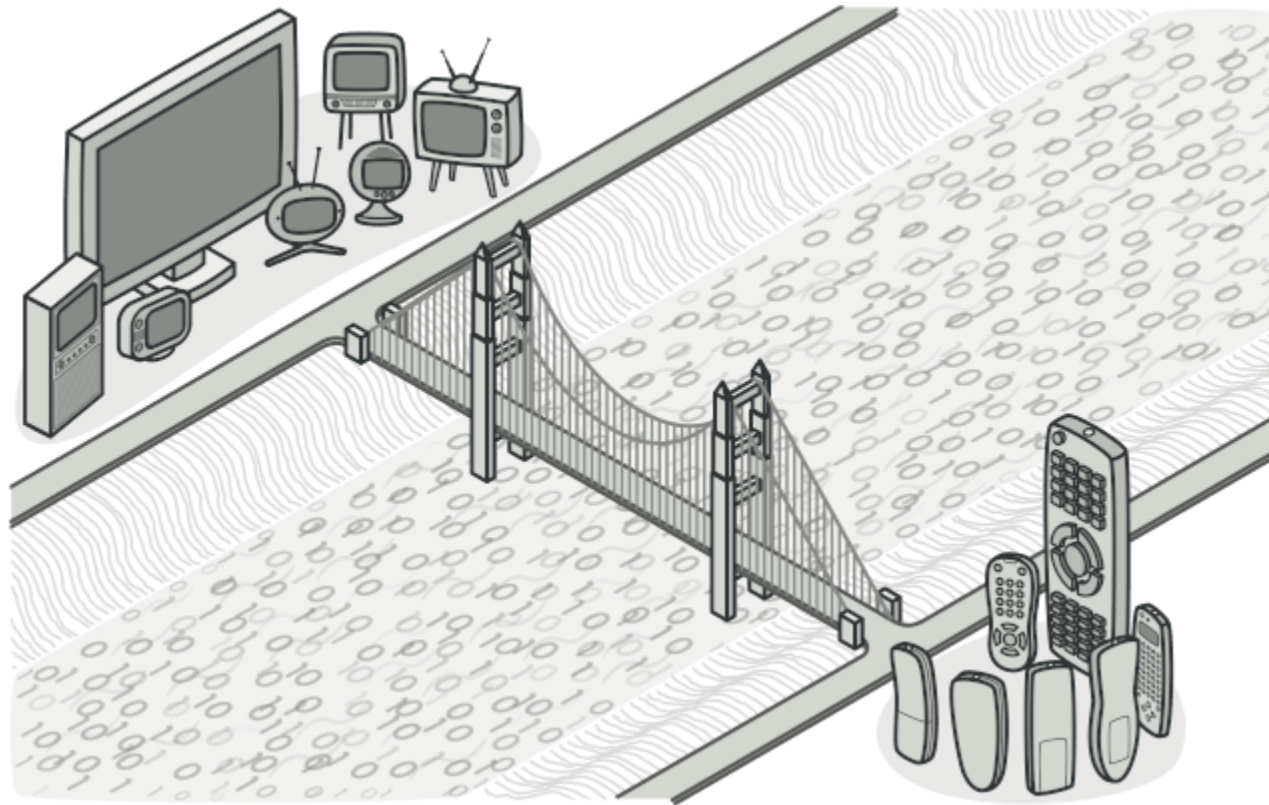
Решение



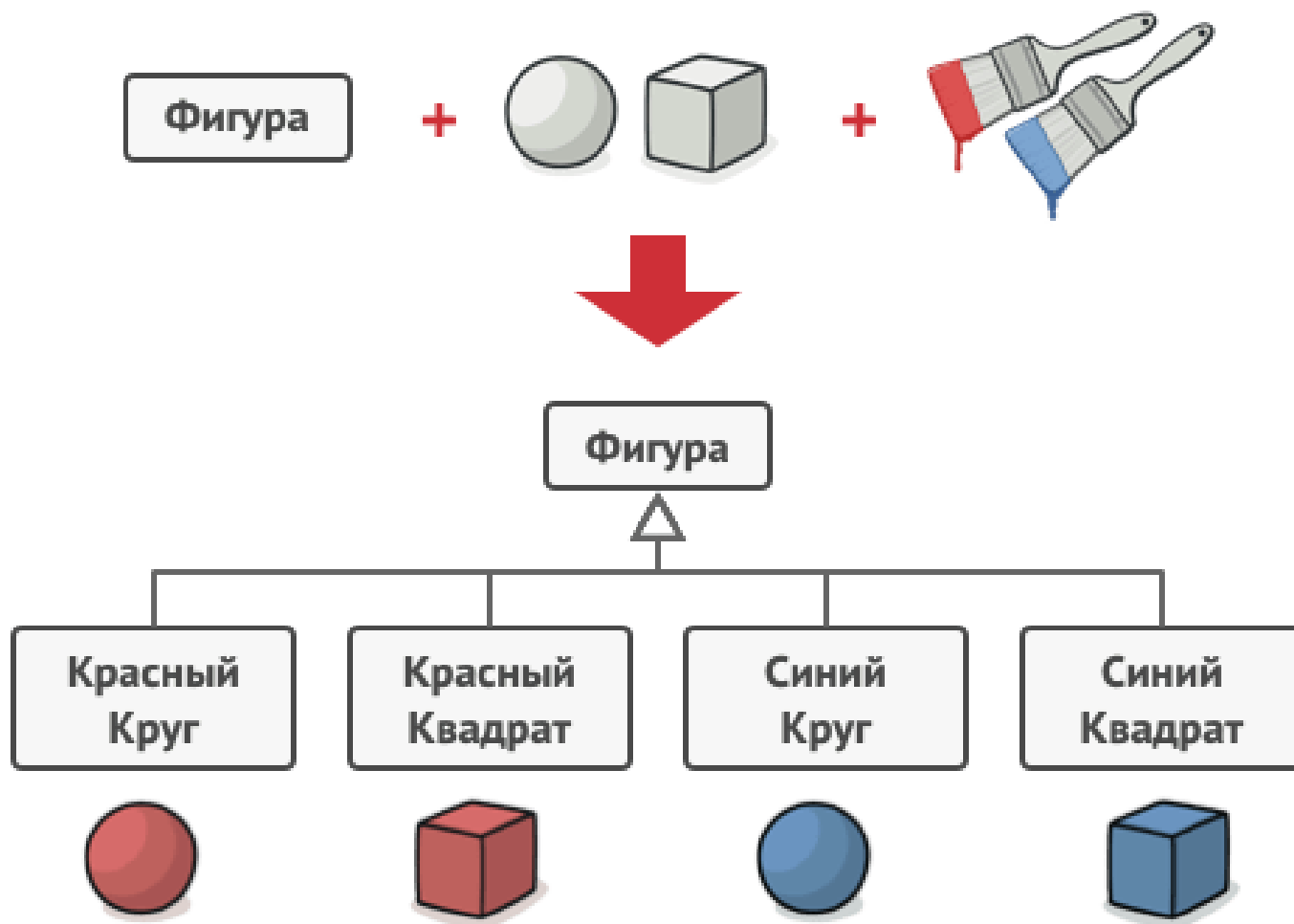
Структура



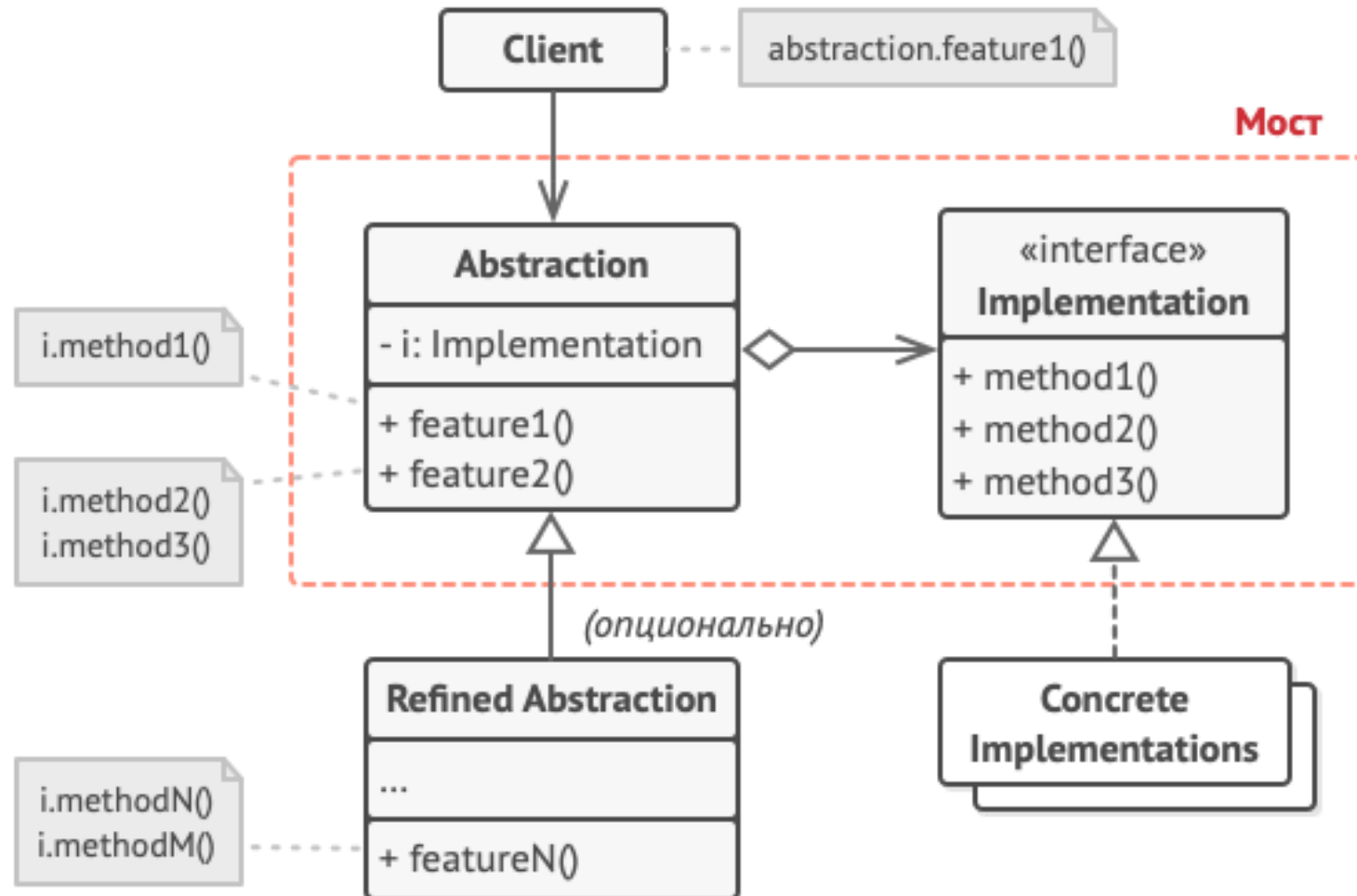
Мост (Bridge)



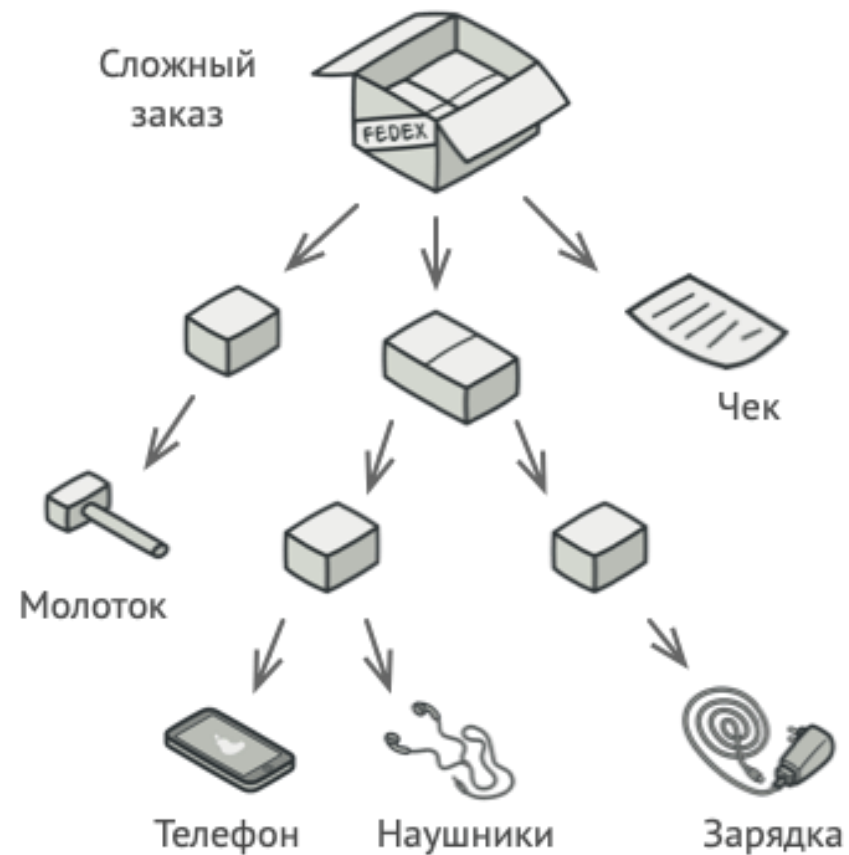
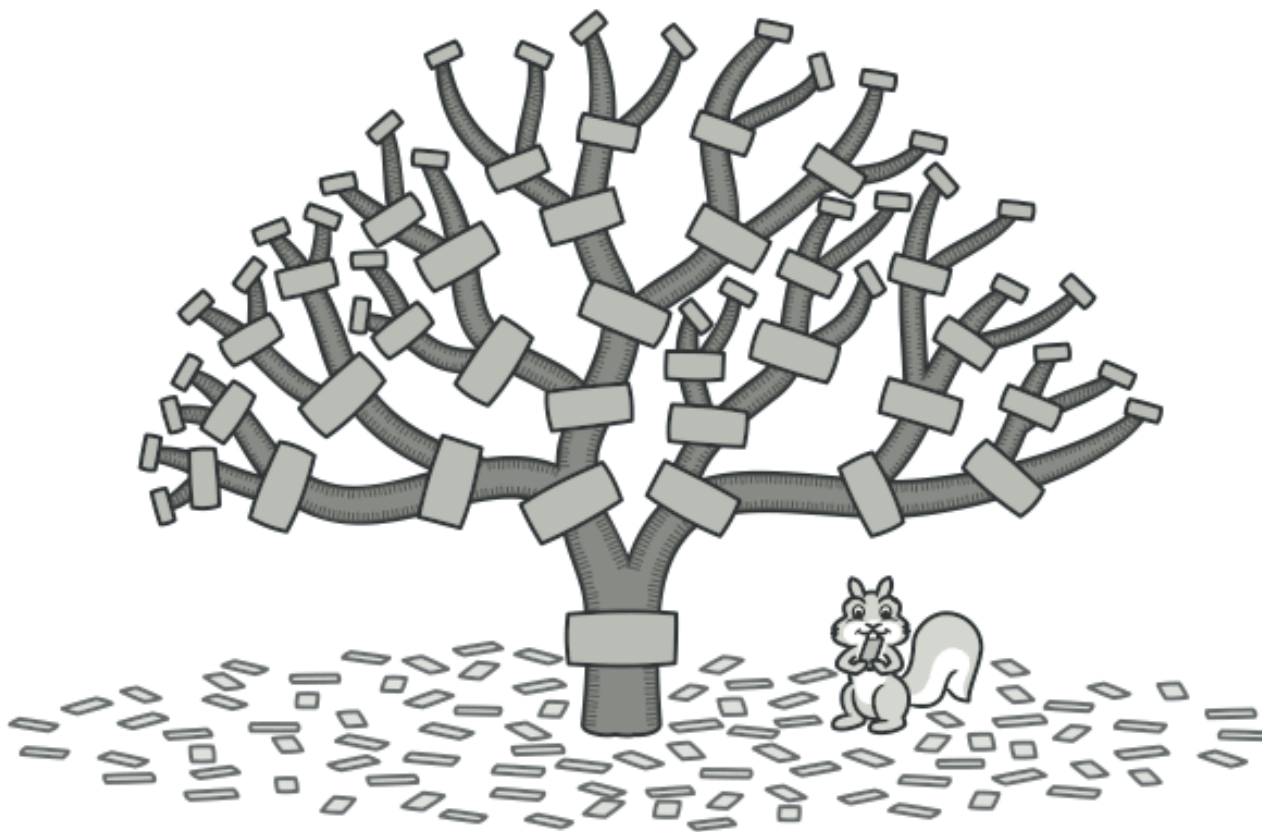
Проблема и решение



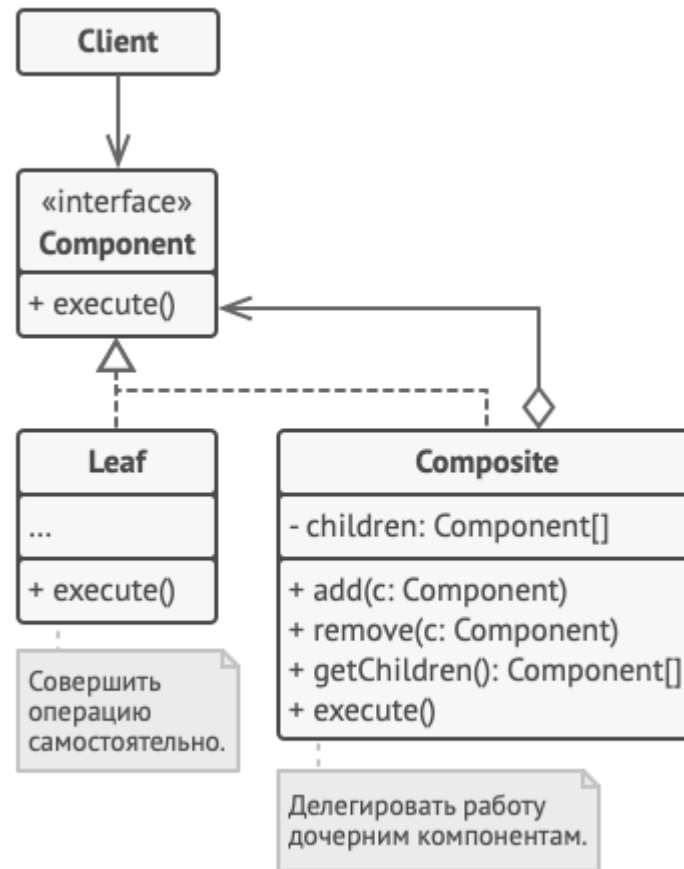
Структура



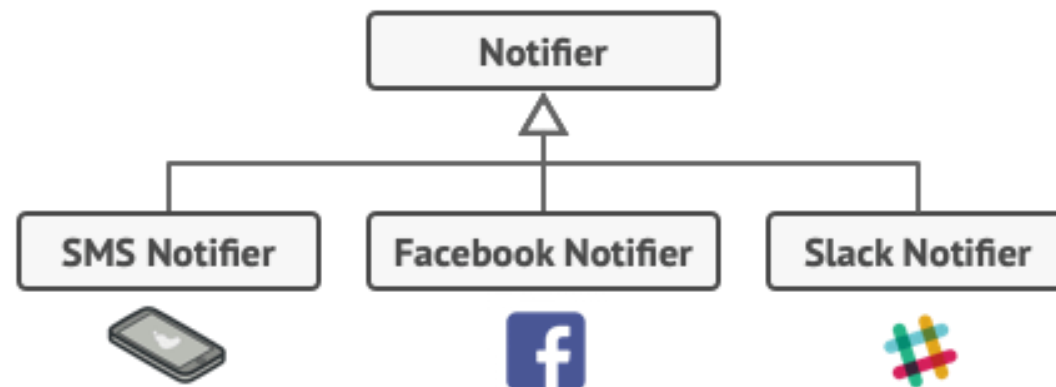
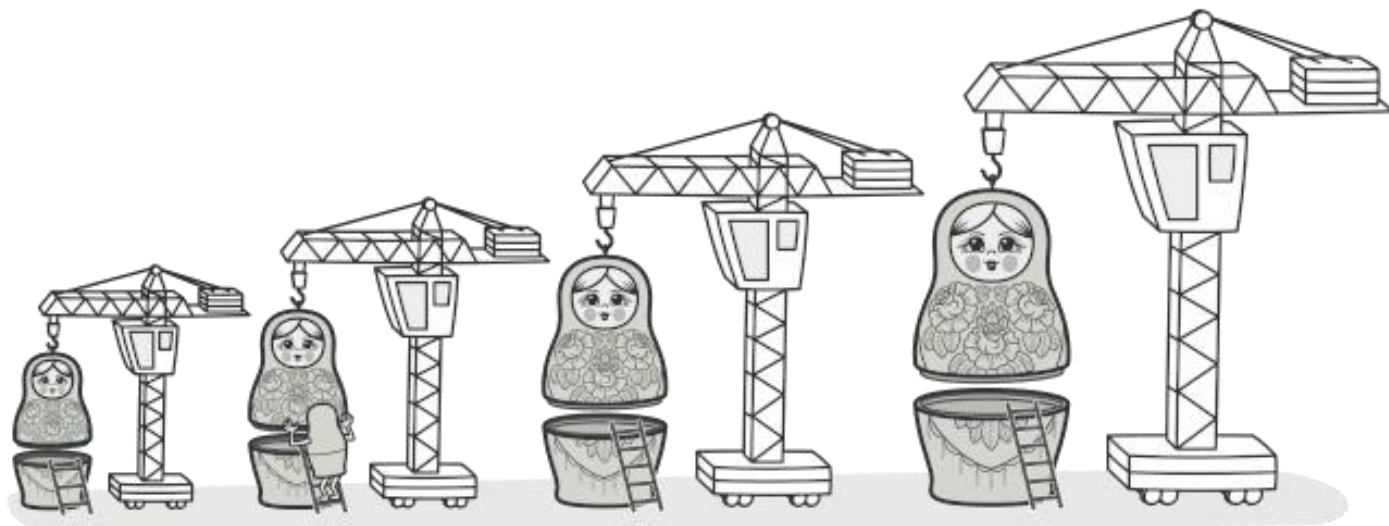
Компоновщик (Composite)



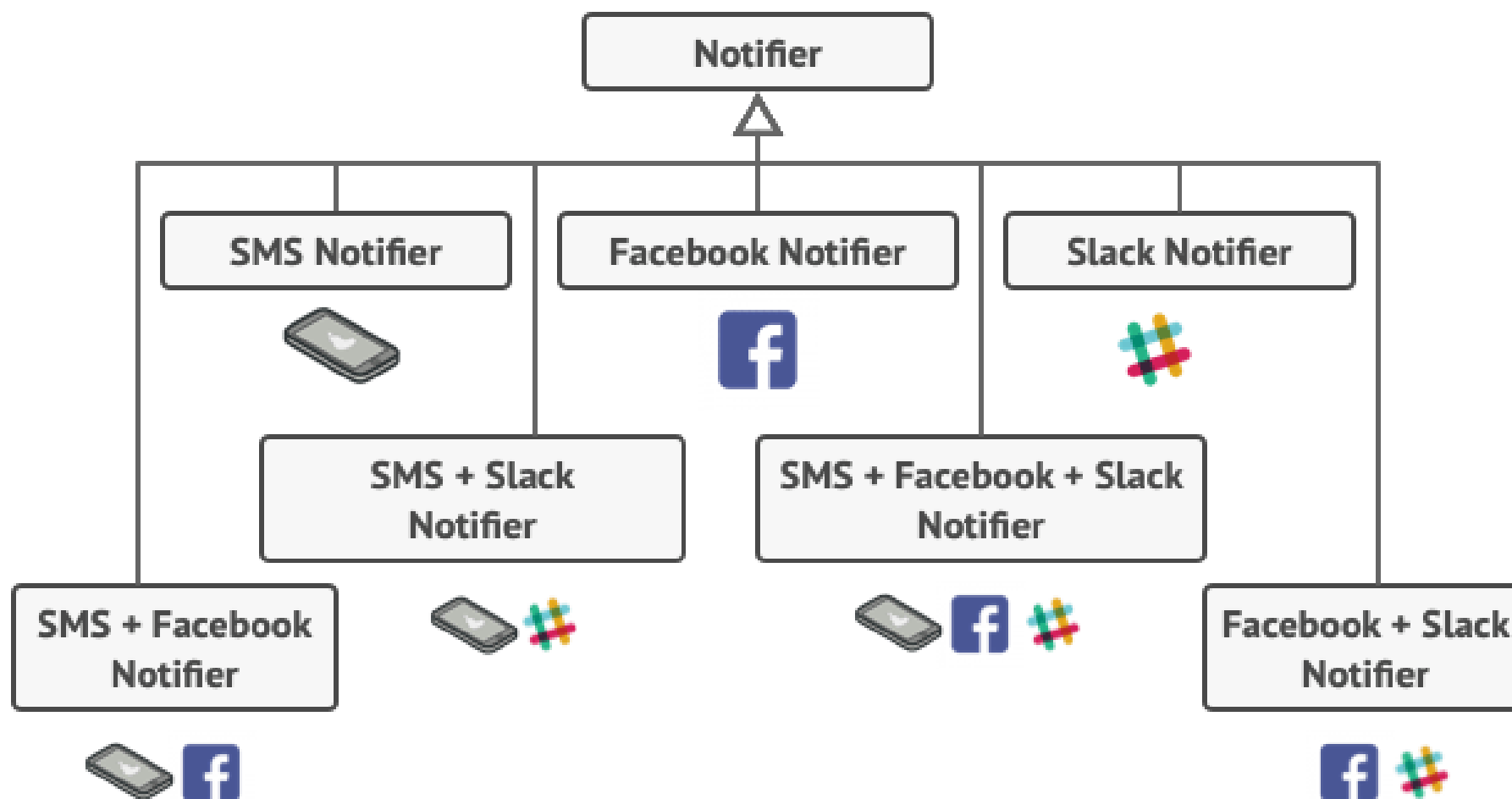
Структура



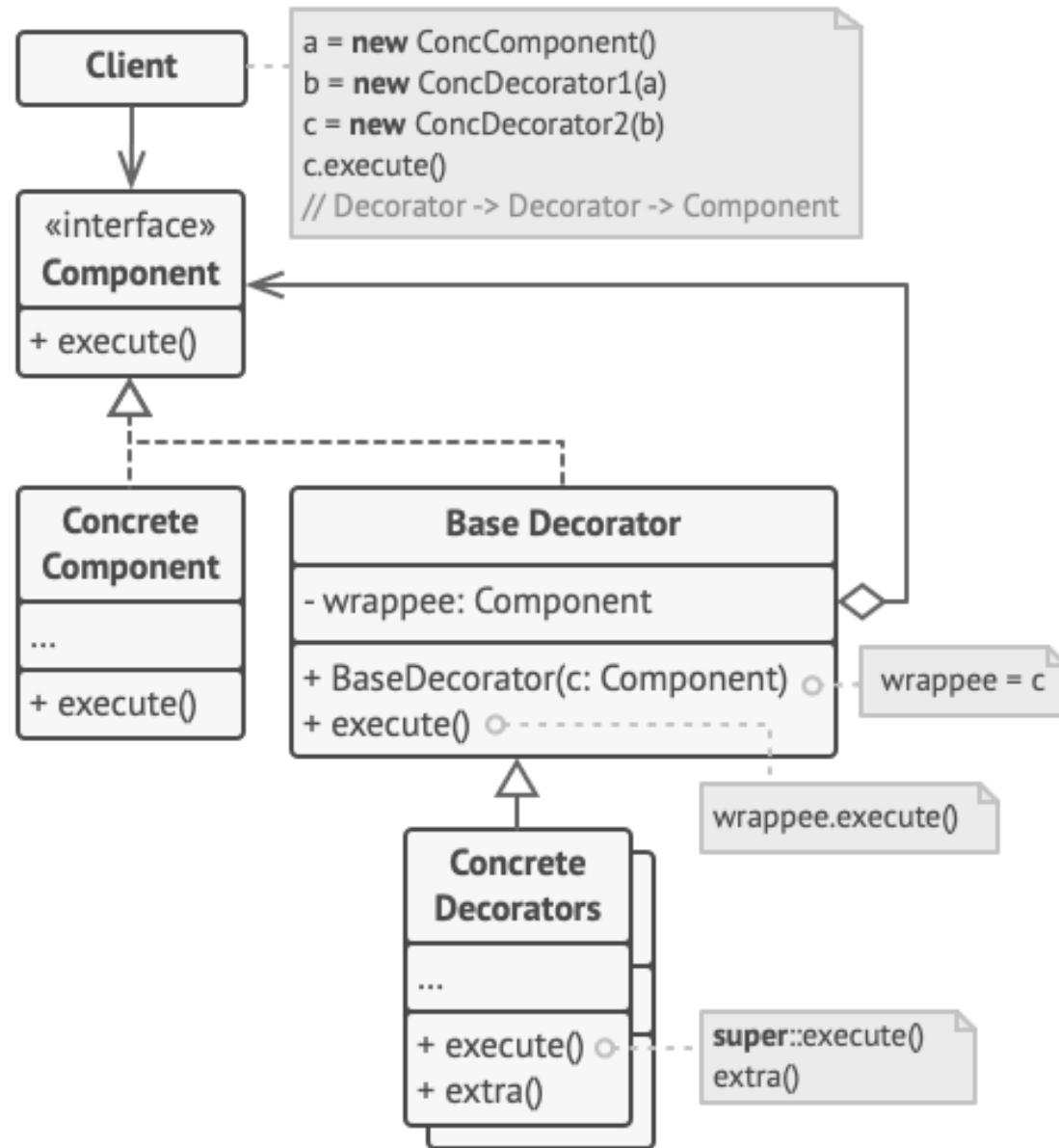
Декоратор (Decorator, Wrapper)



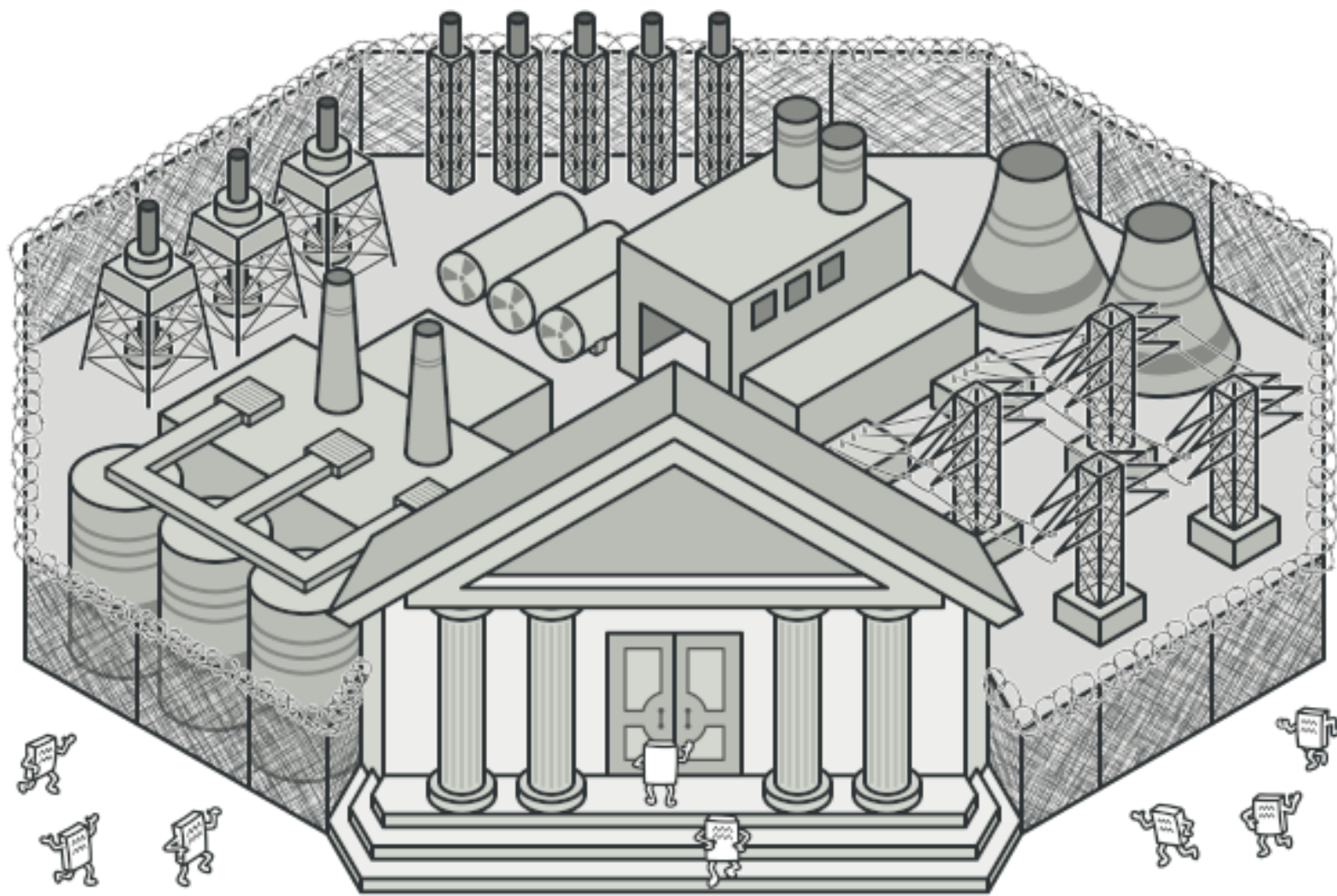
Проблема



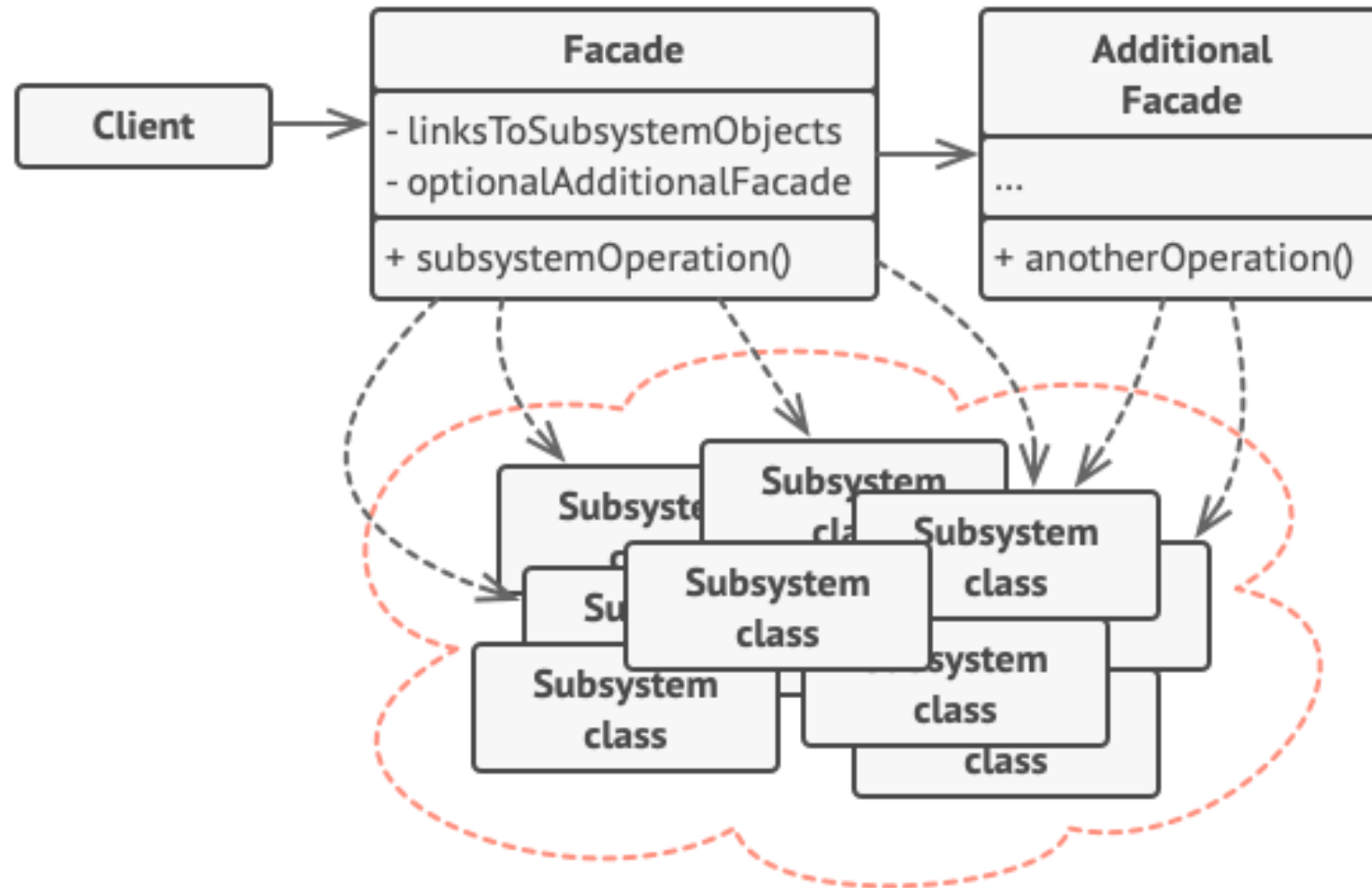
Структура



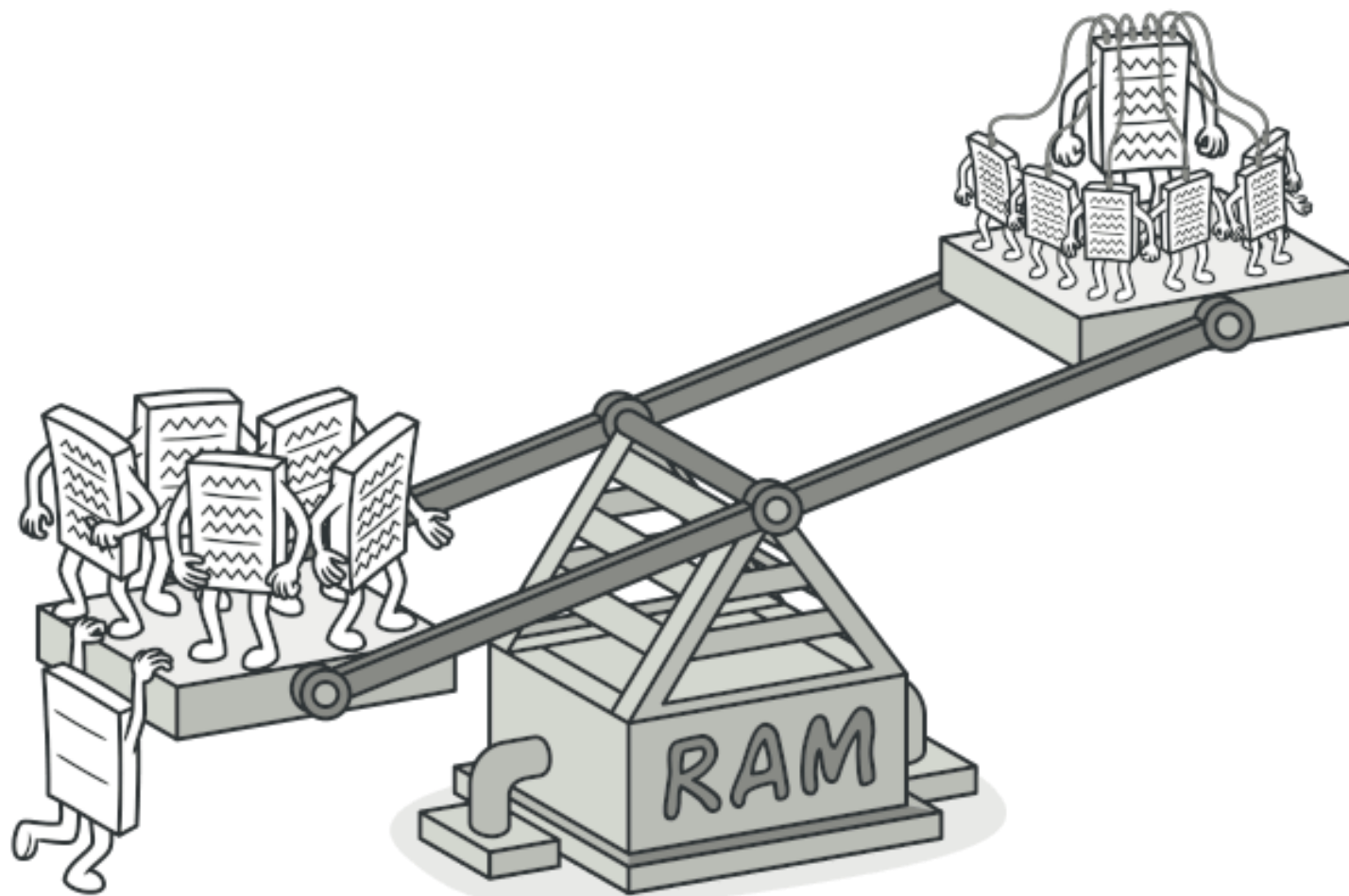
Фасад (Facade)



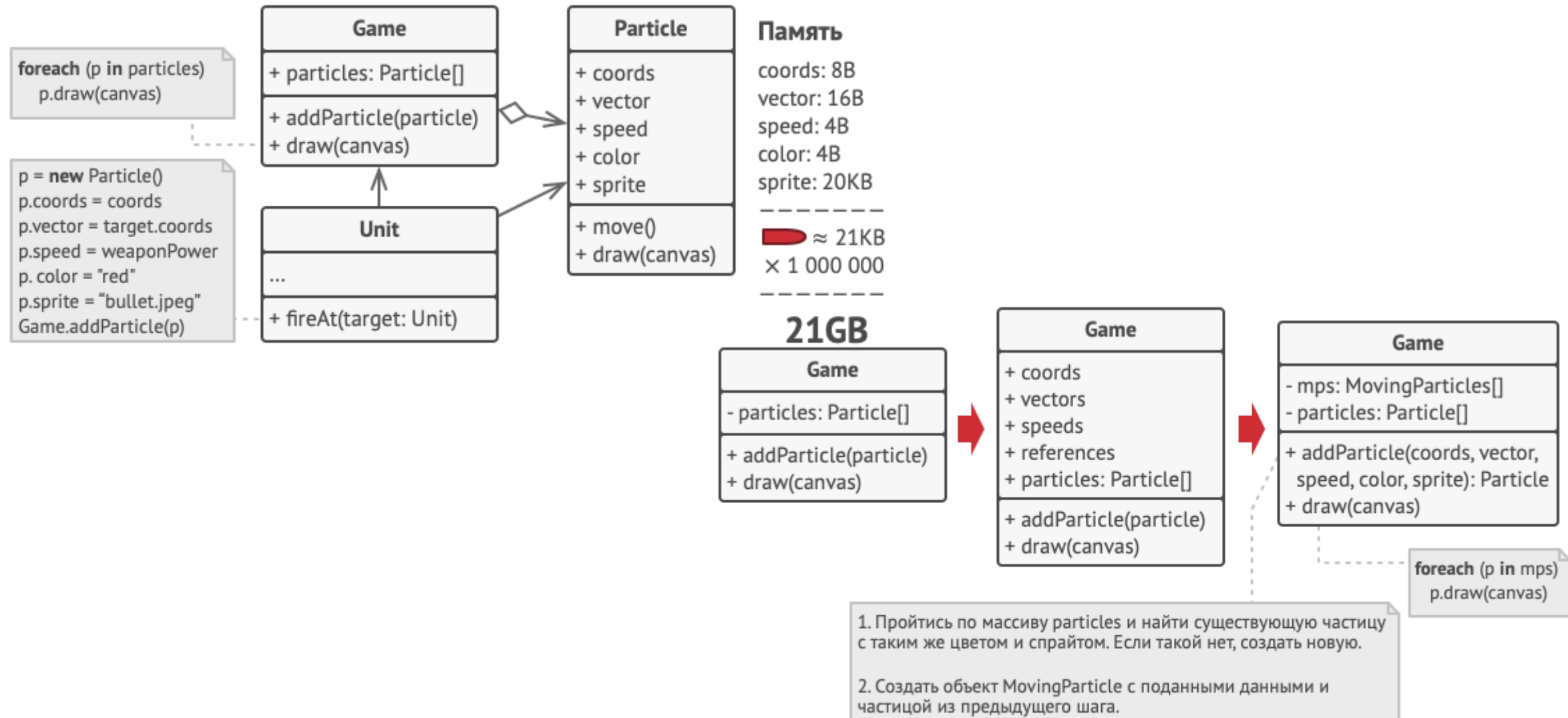
Структура



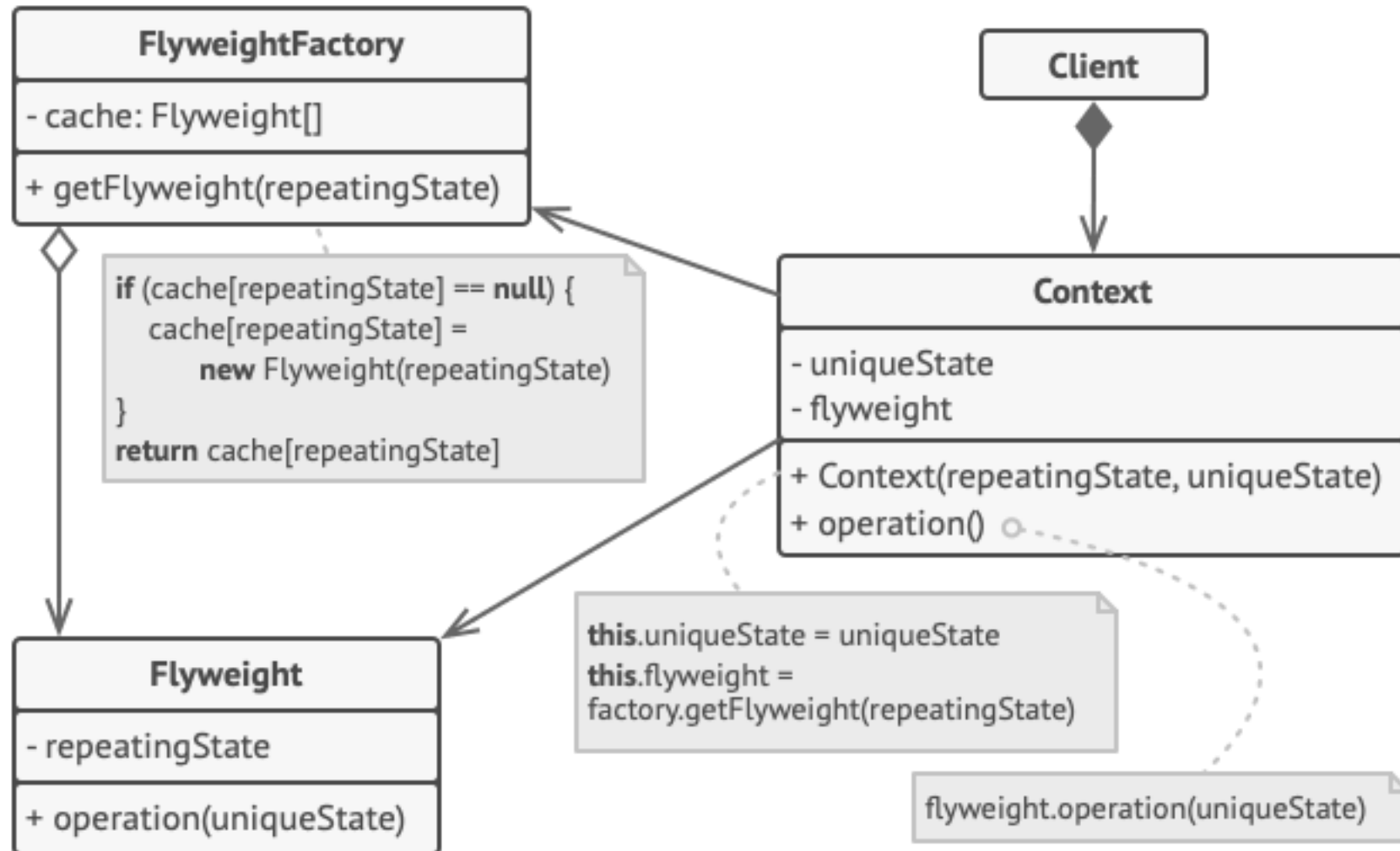
Легковес (Flyweight)



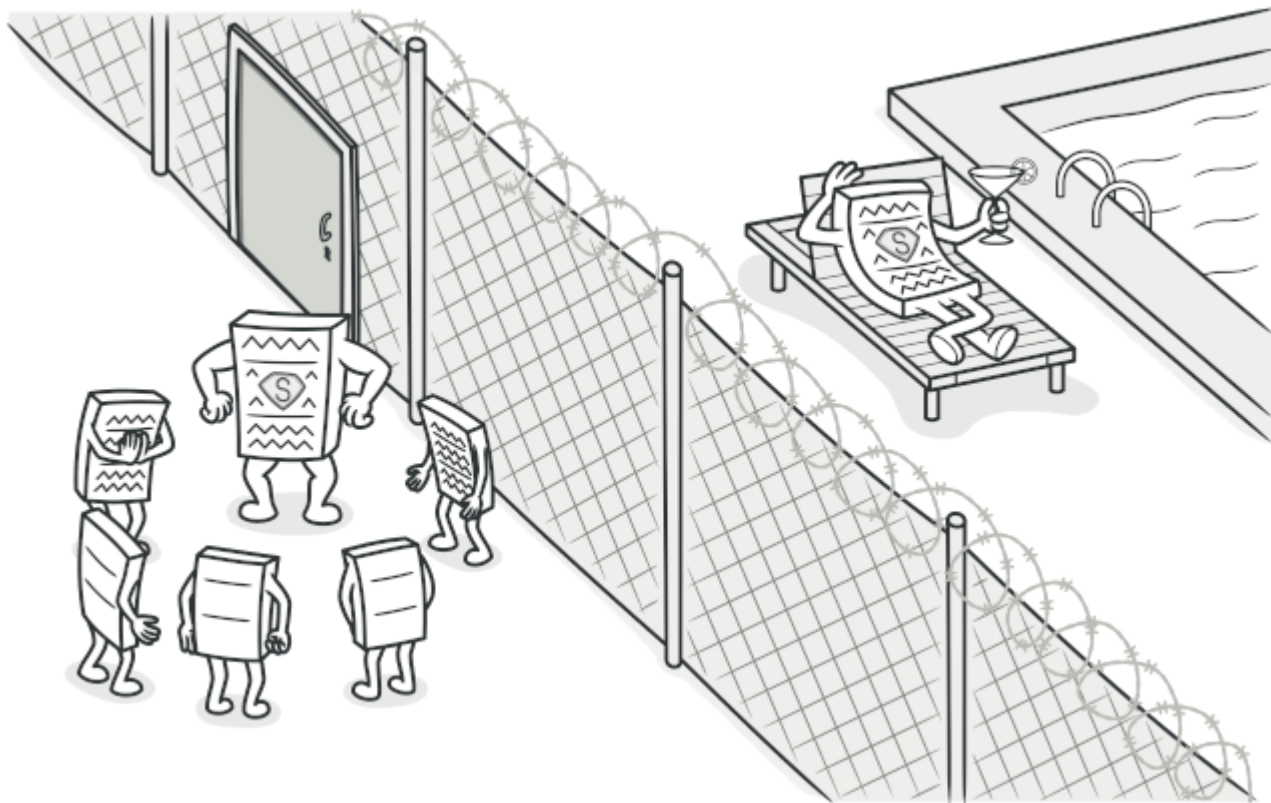
Проблема



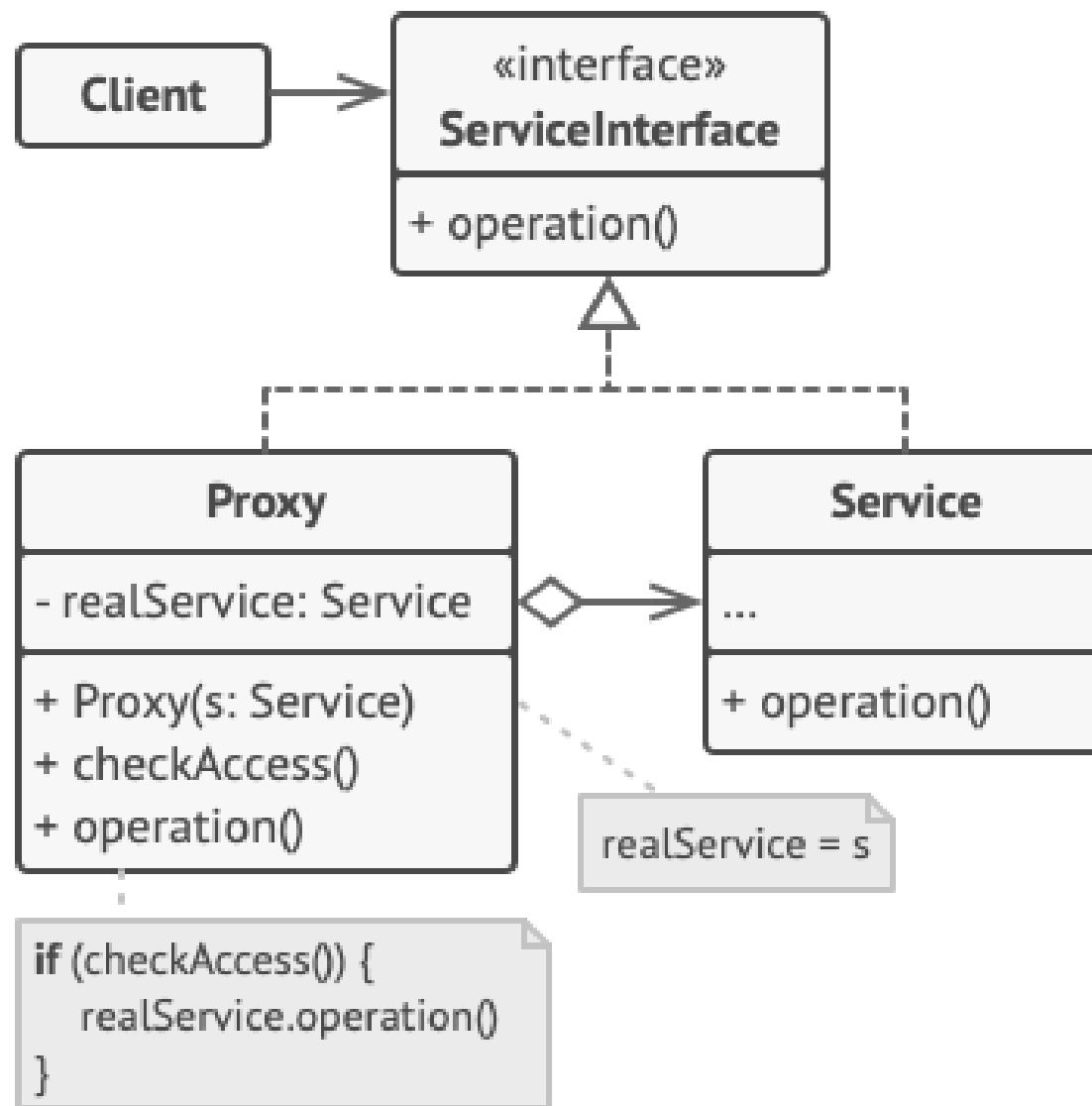
Решение



Заместитель (Proxy)



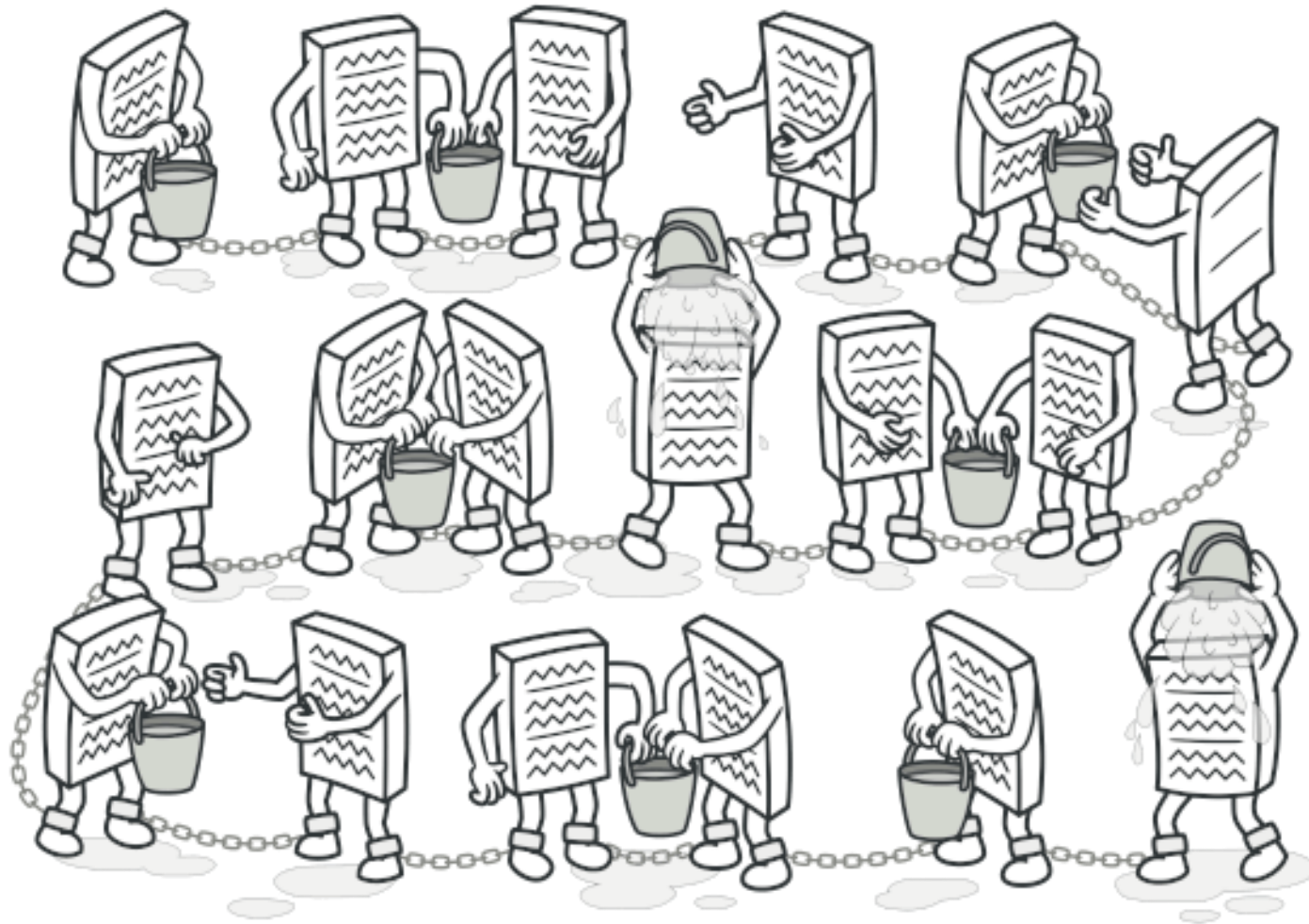
Структура



Поведенческие паттерны проектирования

- **Цепочка обязанностей** (Chain of Responsibility)
- **Команда** (Action)
- **Итератор** (Iterator)
- **Посредник** (Controller, Mediator)
- **Снимок** (Memento)
- **Наблюдатель** (Observer)
- **Состояние** (State)
- **Стратегия** (Strategy)
- **Шаблонный метод** (Template Method)
- **Посетитель** (Visitor)

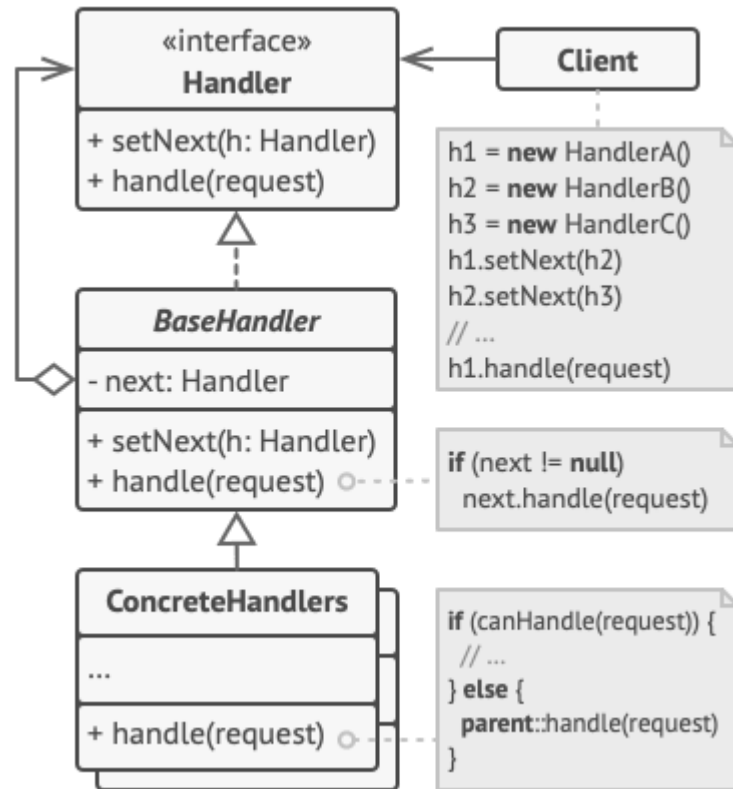
Цепочка обязанностей (Chain of Responsibility)



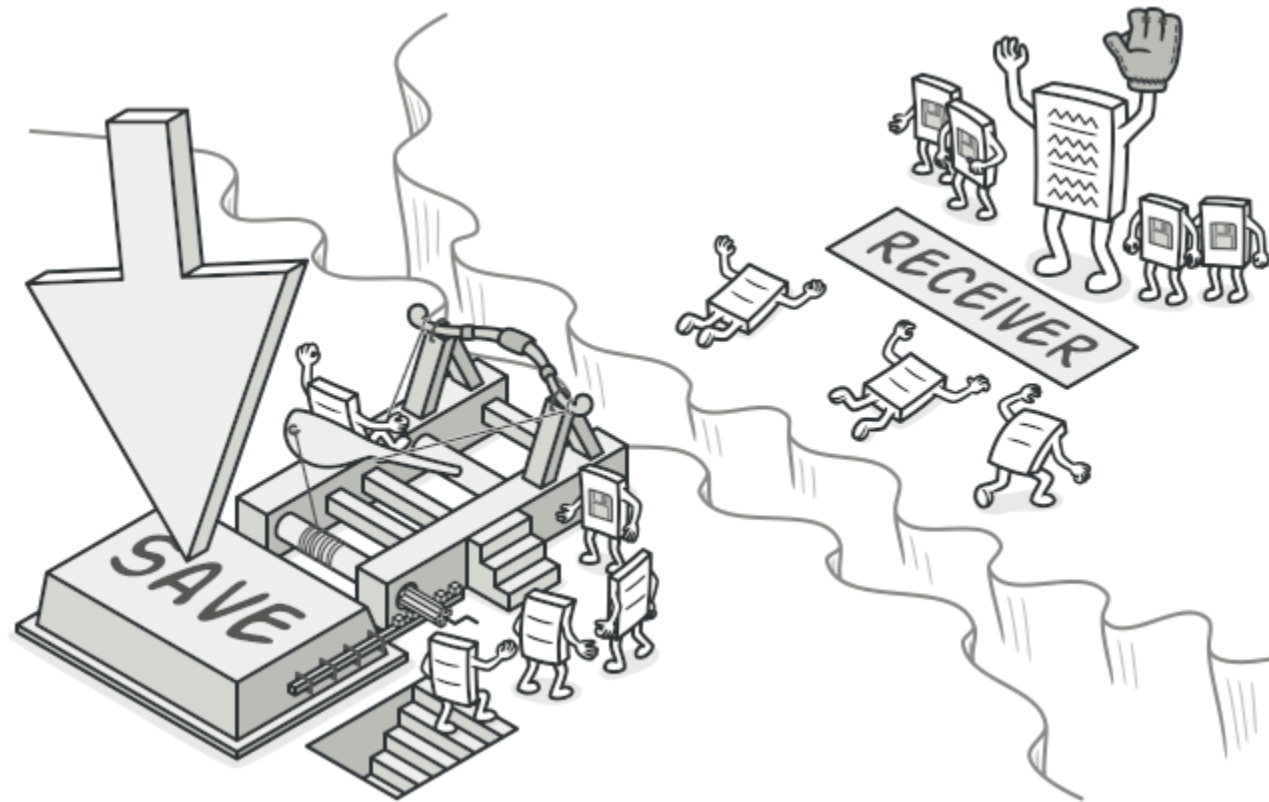
Проблема

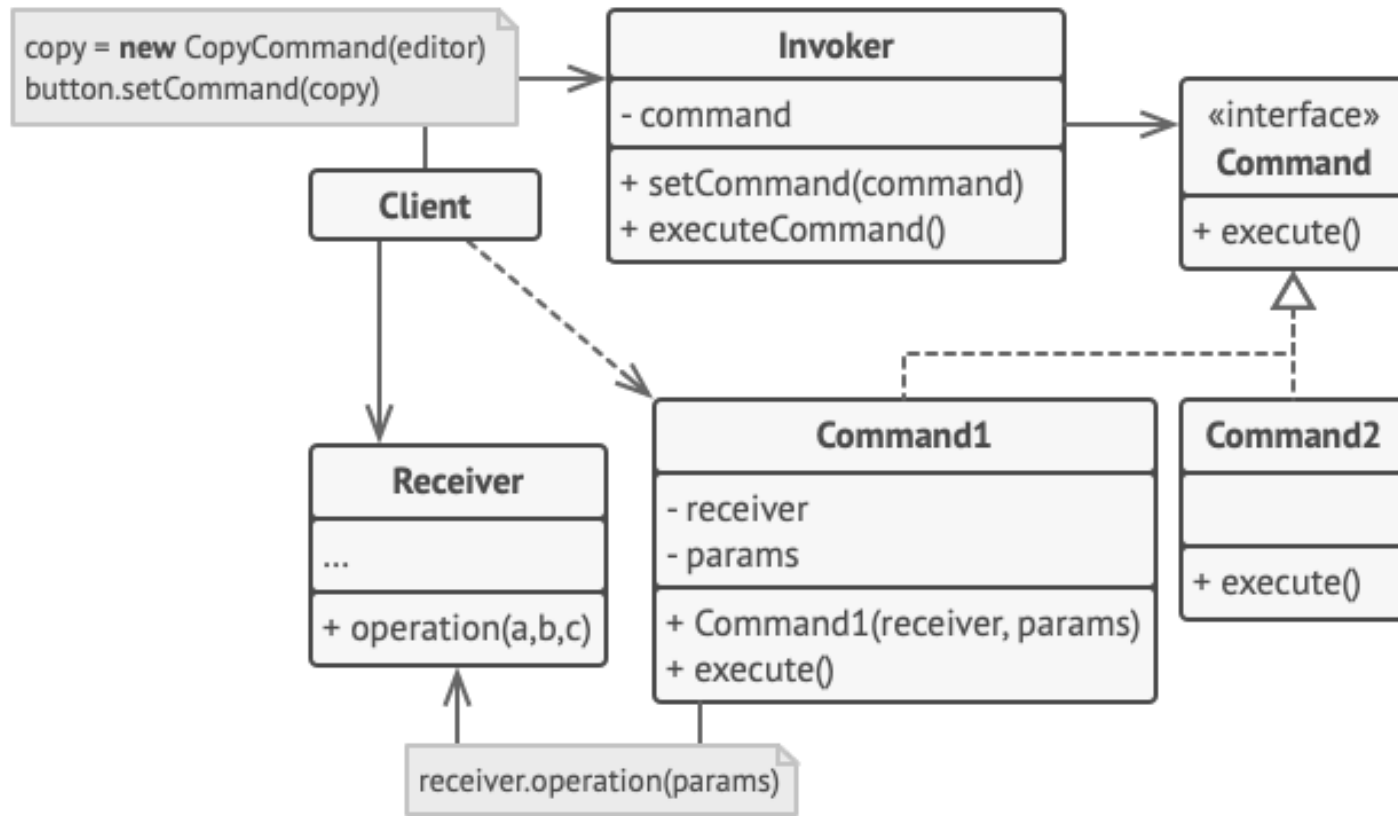


Структура

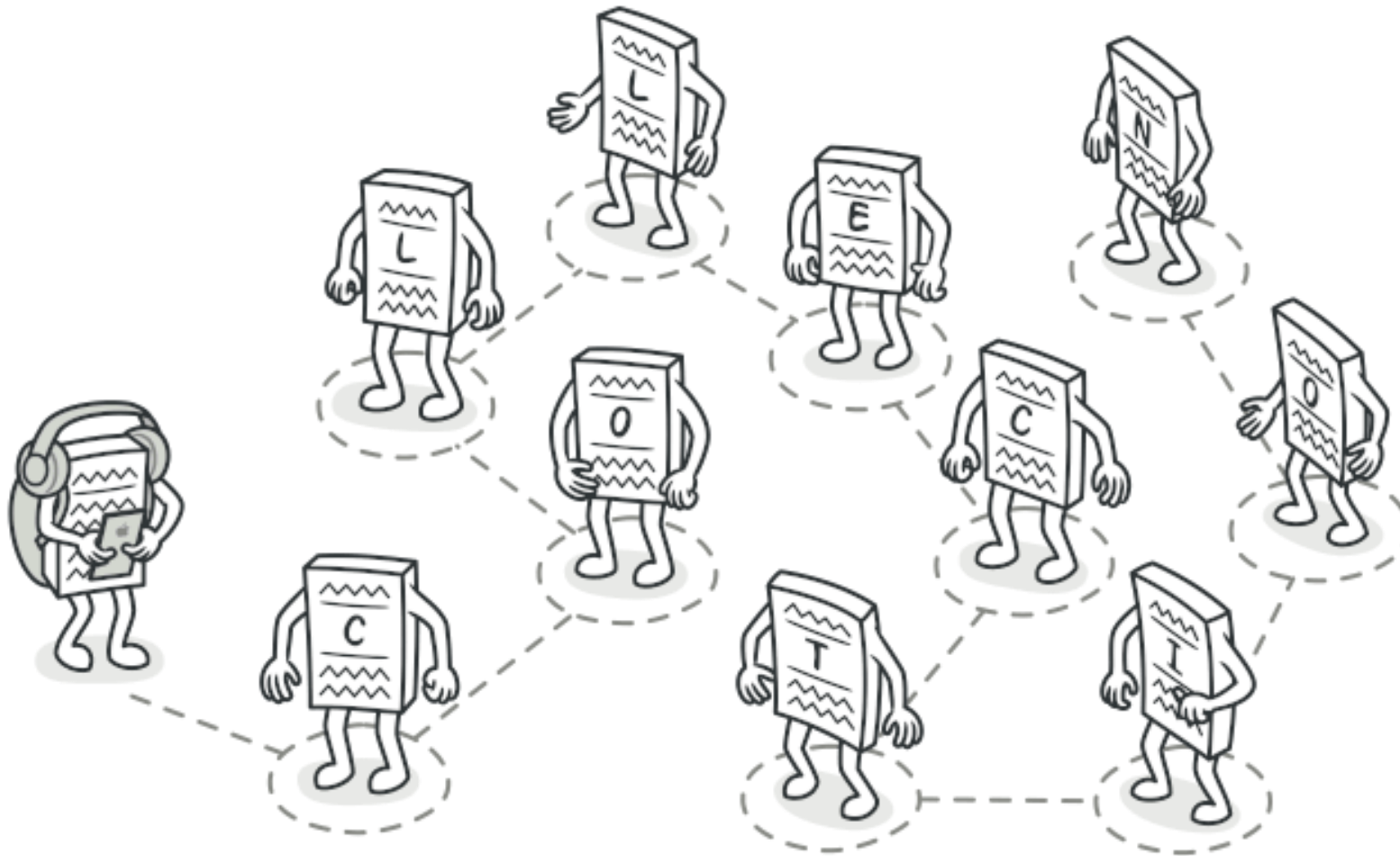


Команда (Action)

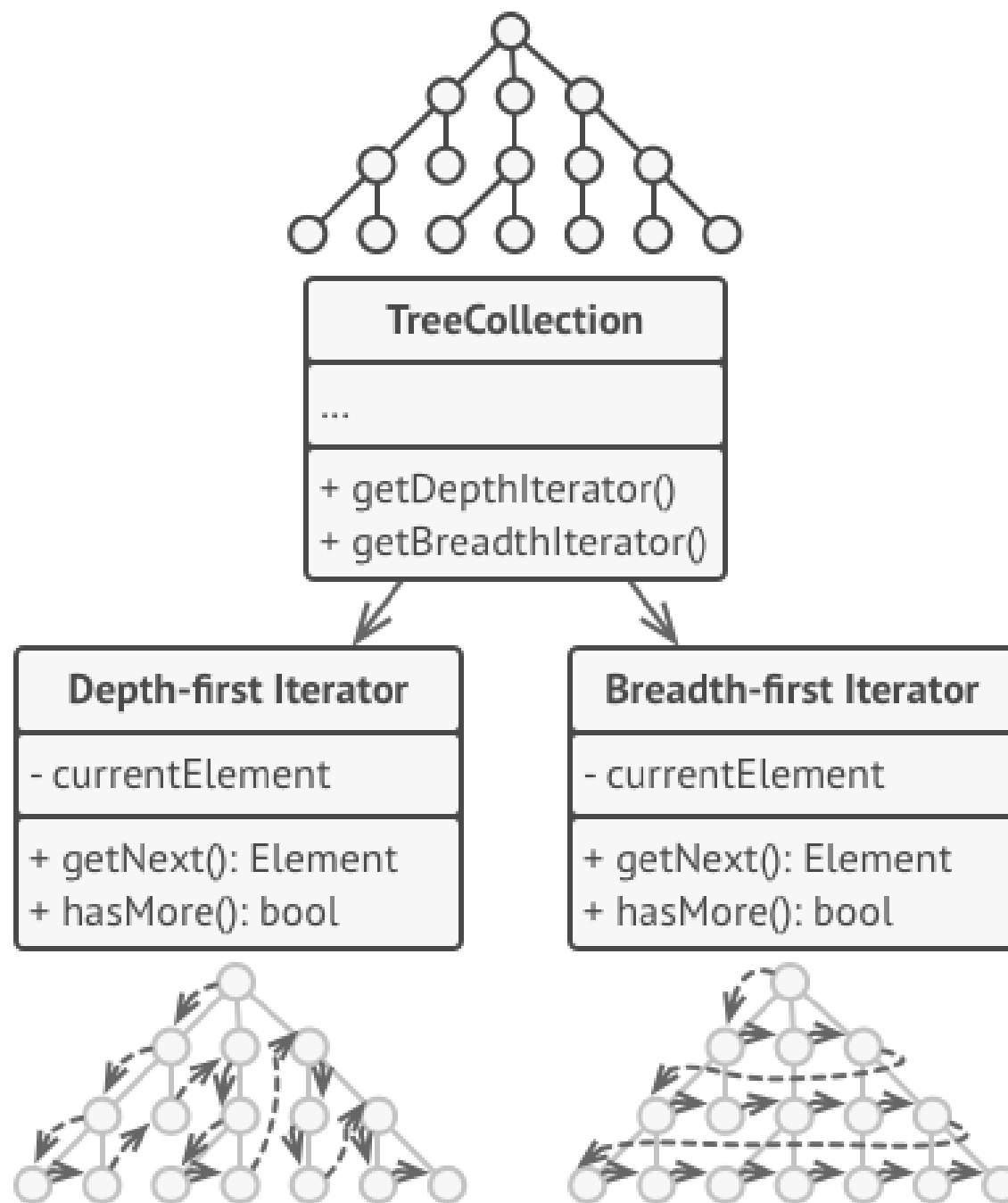




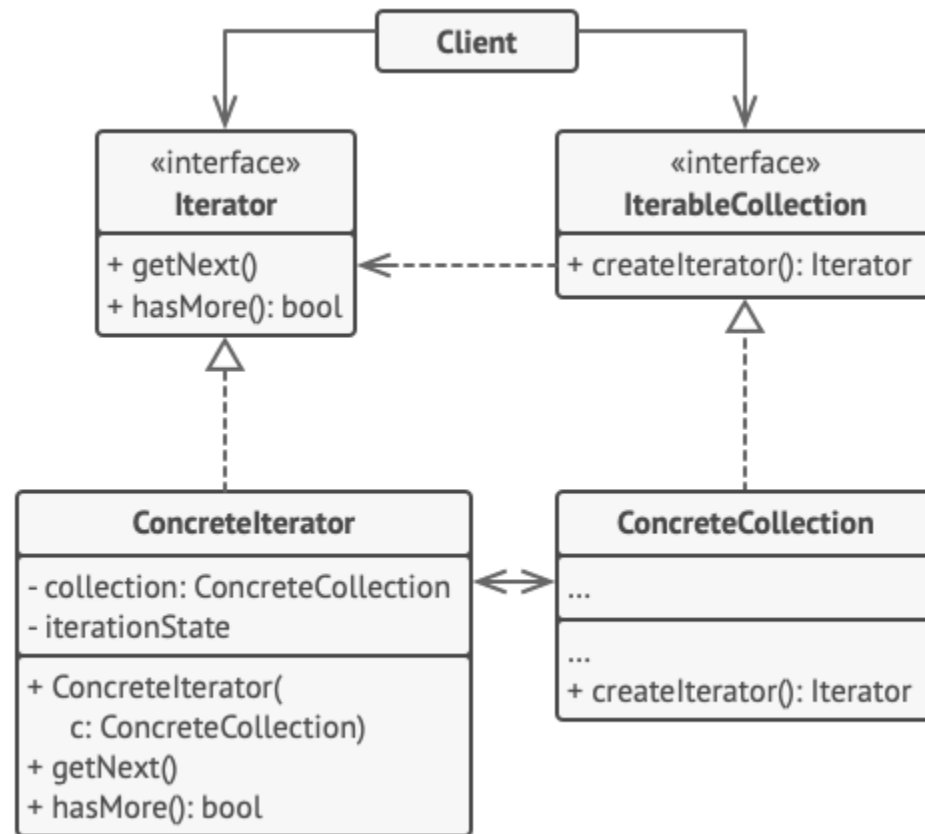
Итератор (Iterator)



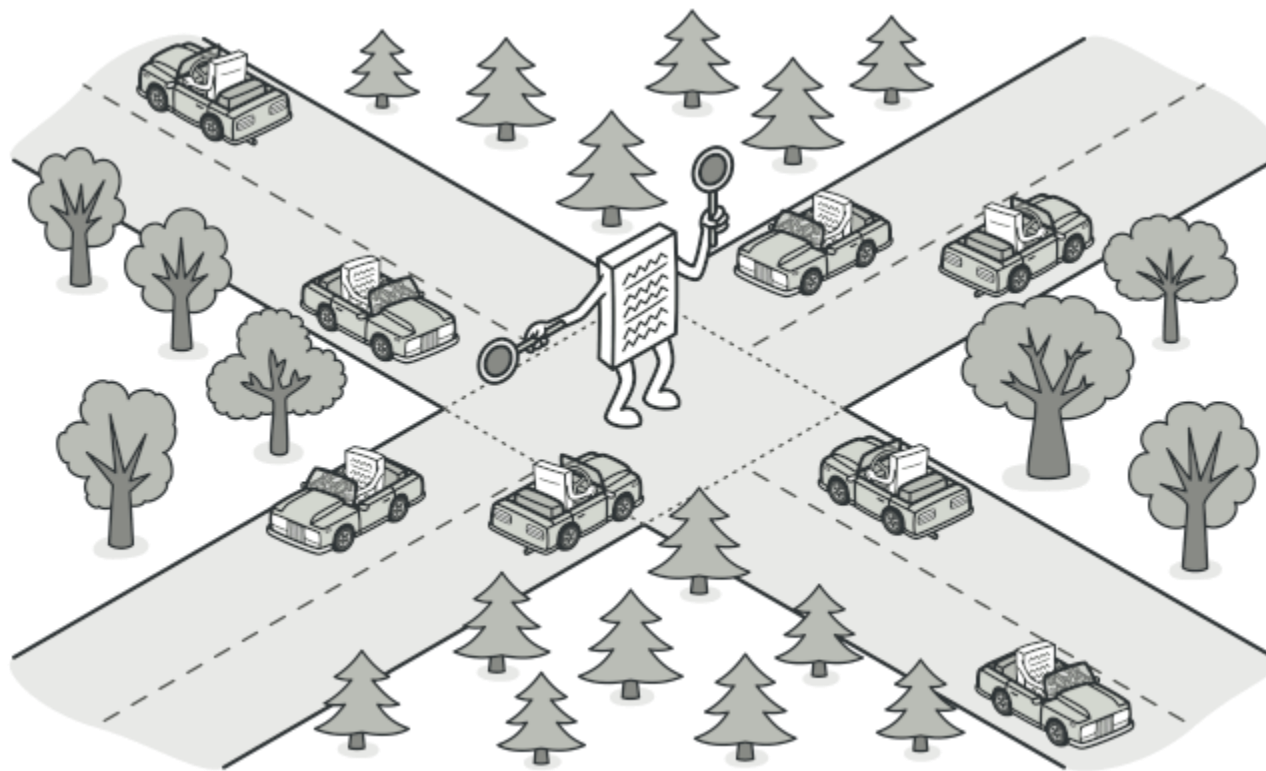
Проблема



Структура

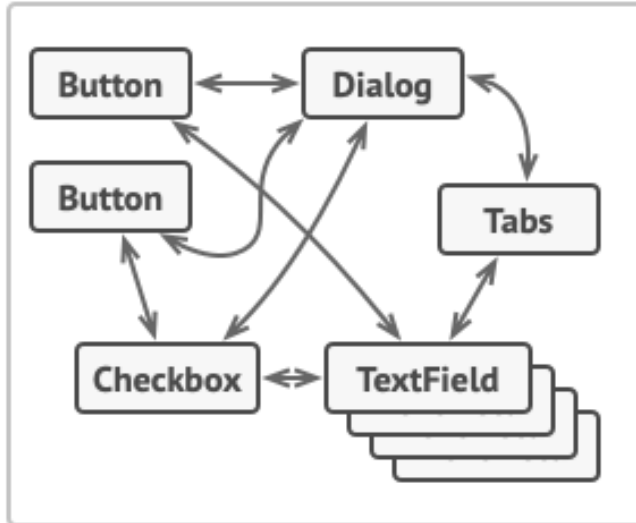


Посредник (Controller, Mediator)

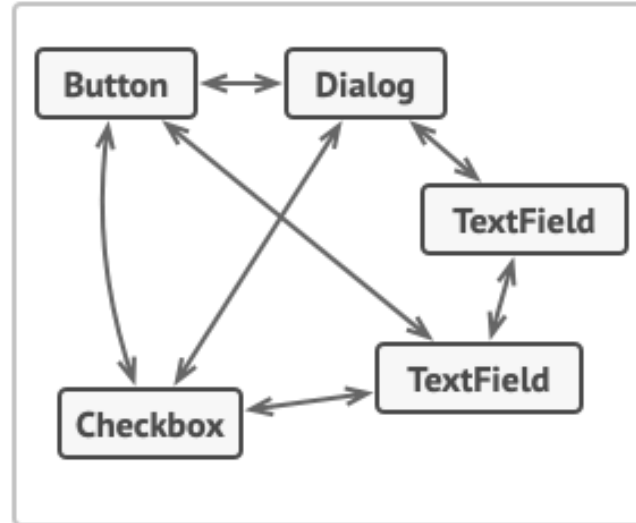


Проблема

Profile Dialog

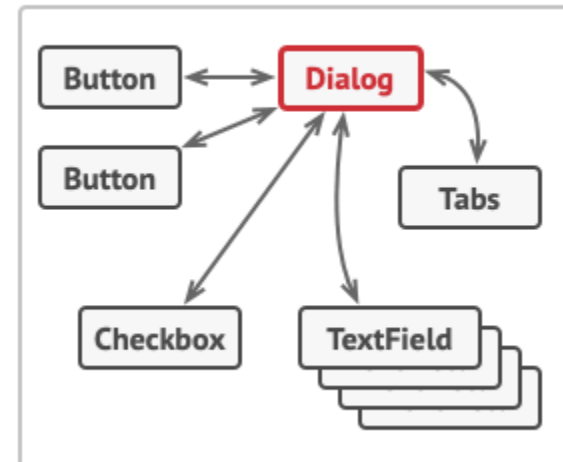


Login Dialog

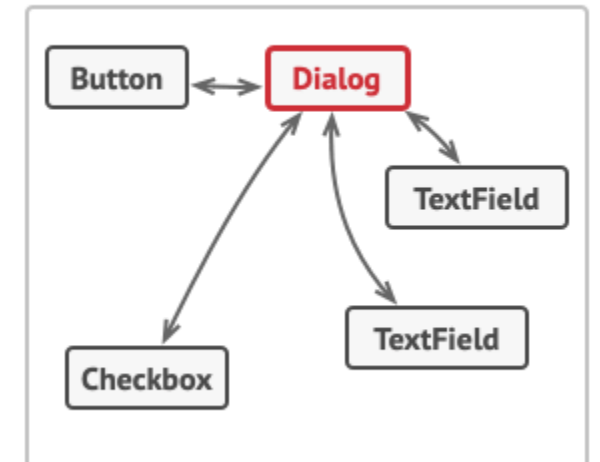


Беспорядочные связи между элементами
пользовательского интерфейса.

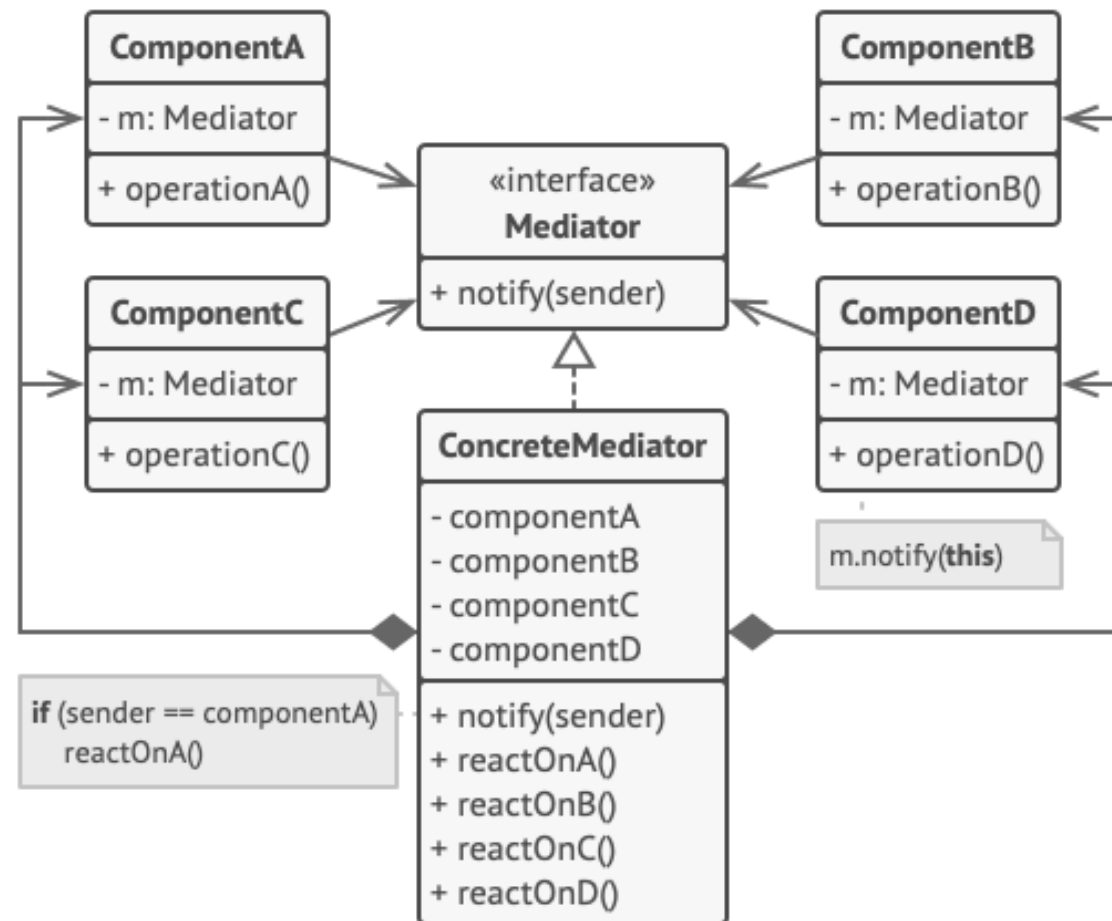
Profile Dialog



Login Dialog



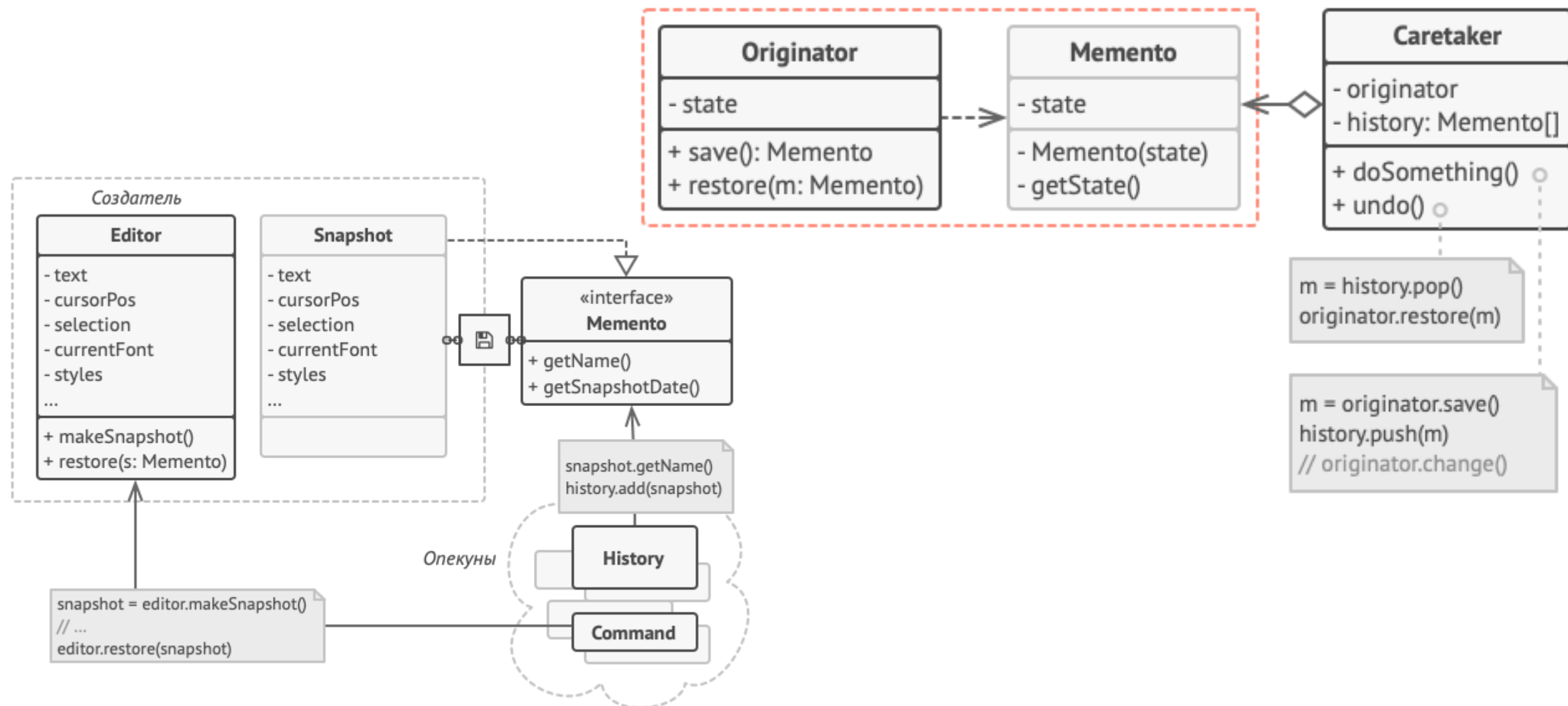
Структура



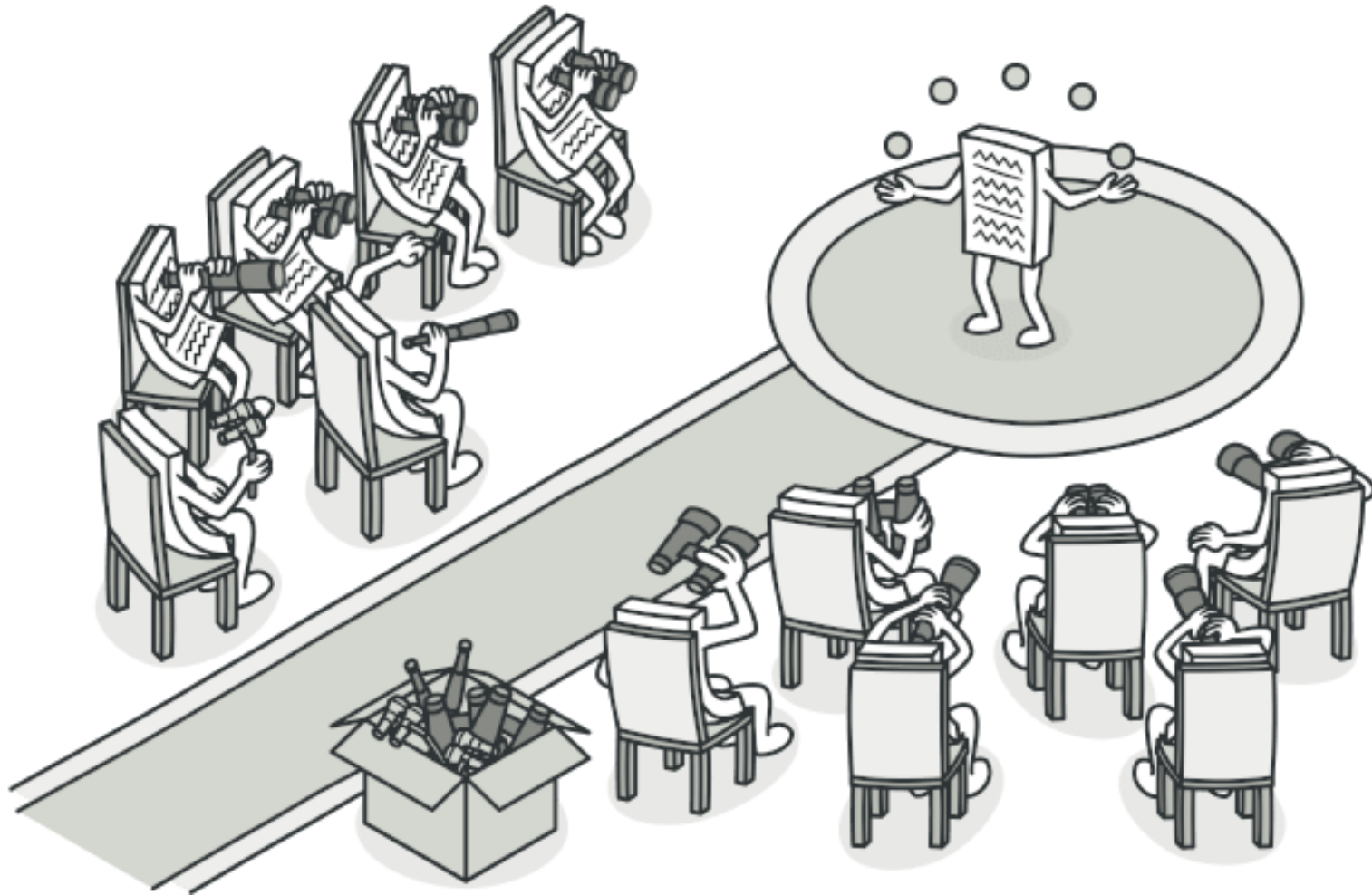
Снимок (Memento)



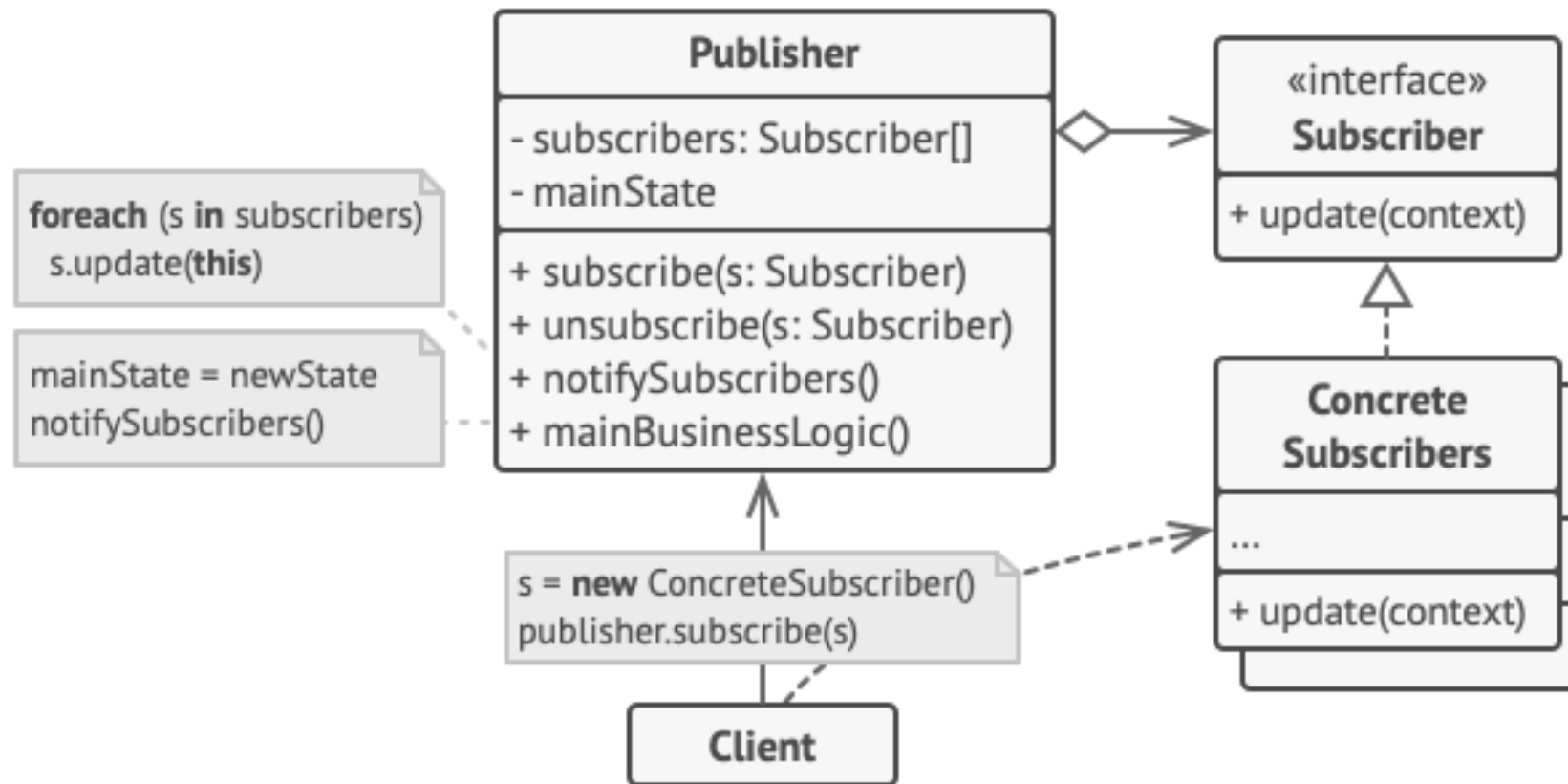
Структура



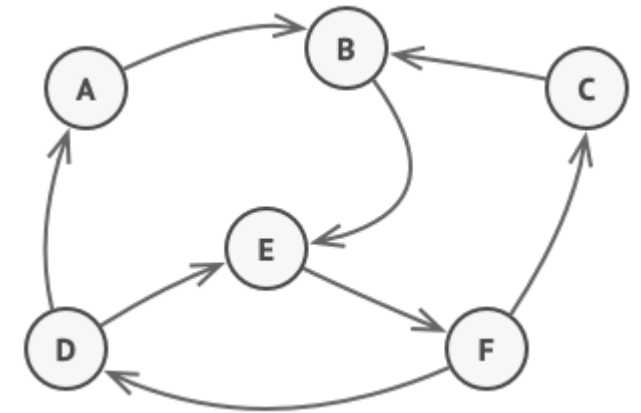
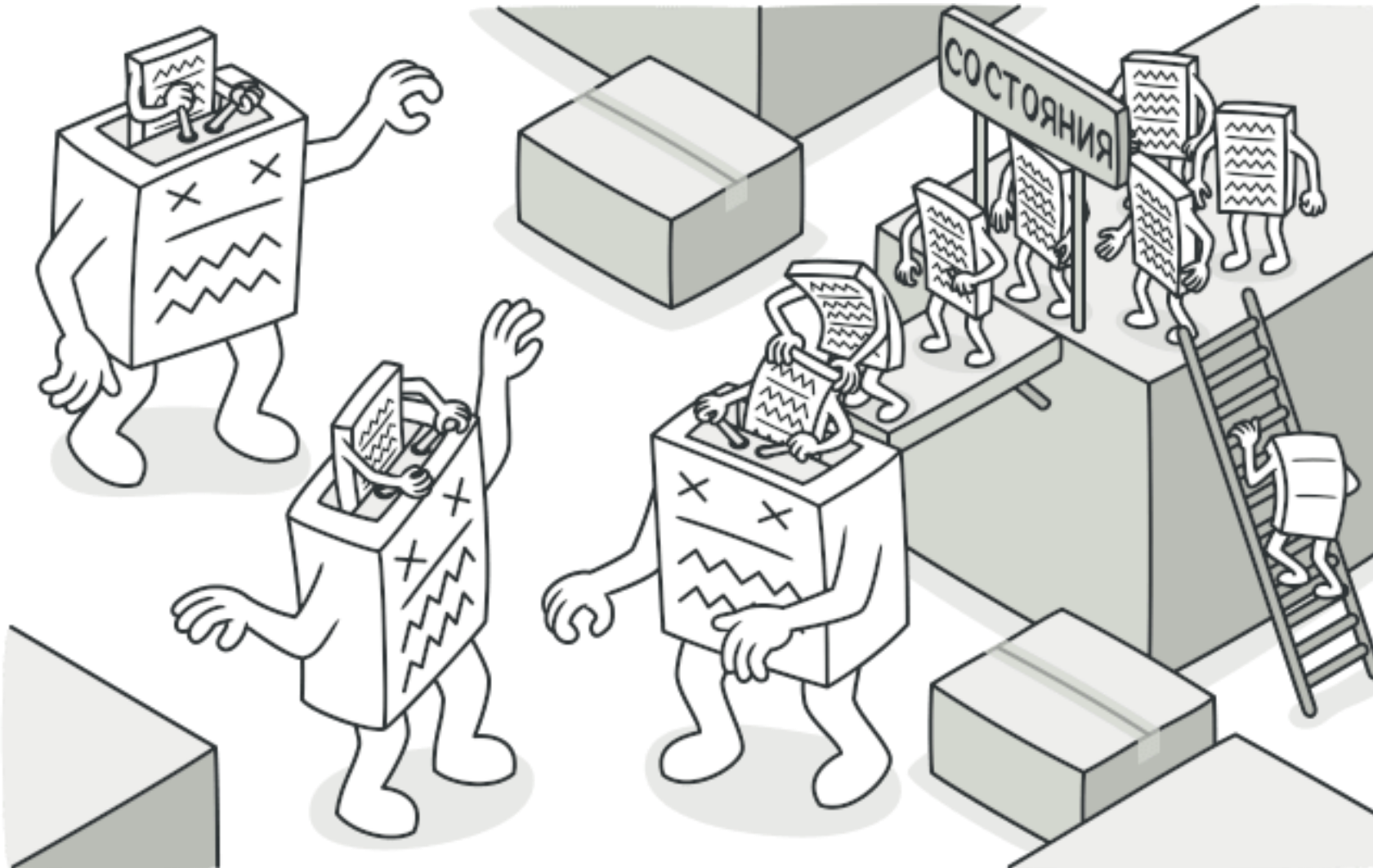
Наблюдатель (Observer)



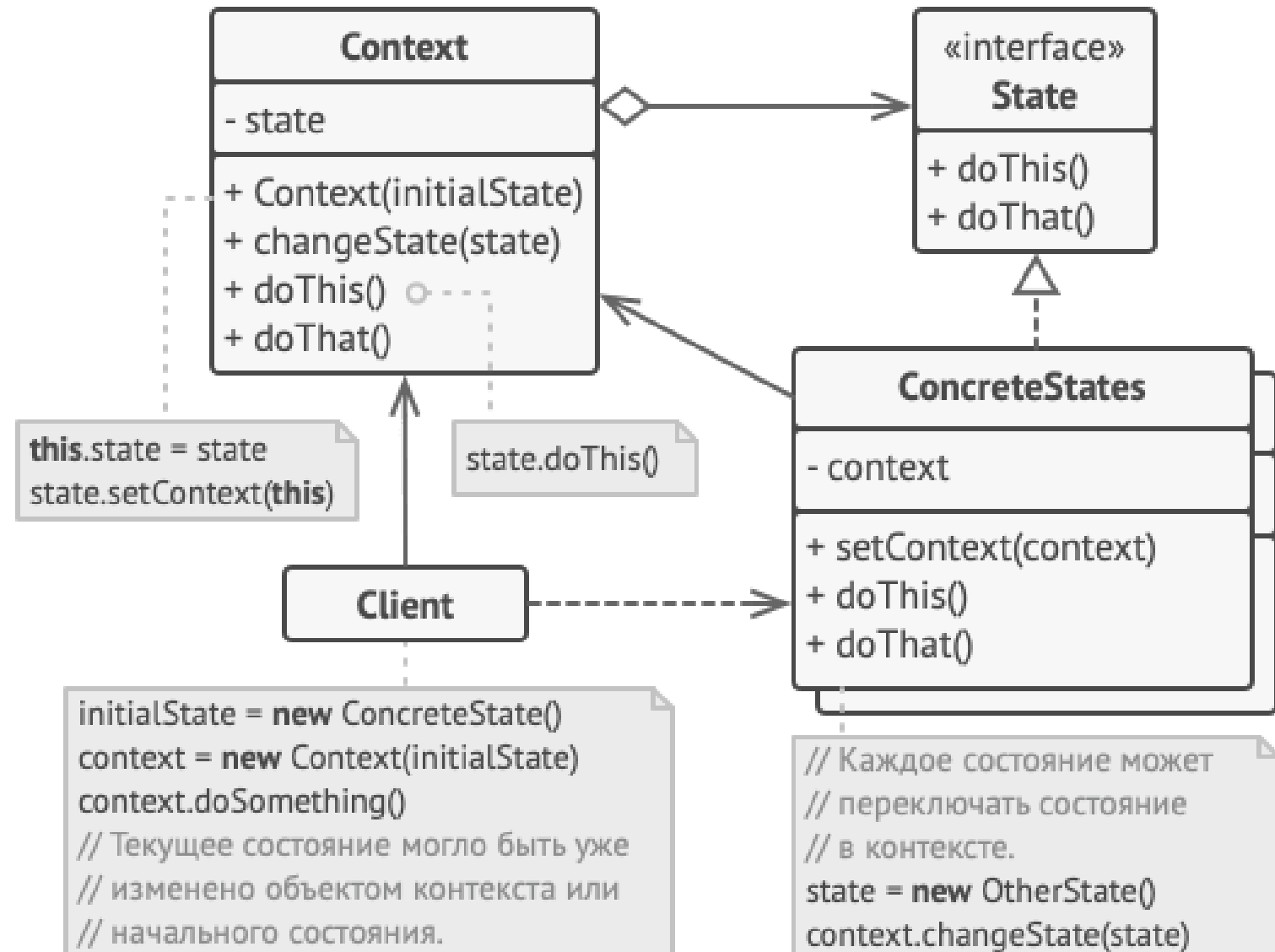
Структура



Состояние (State)



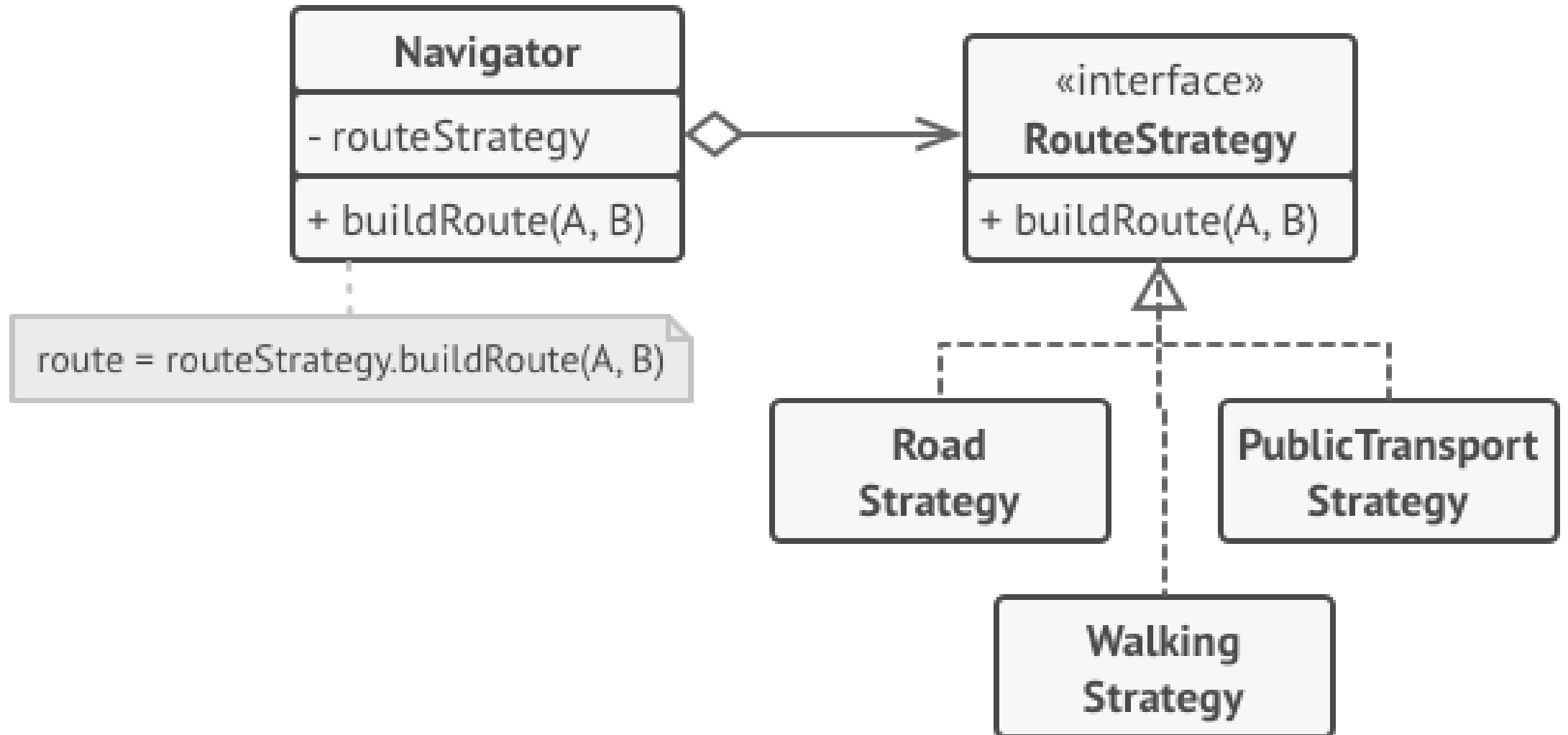
Структура



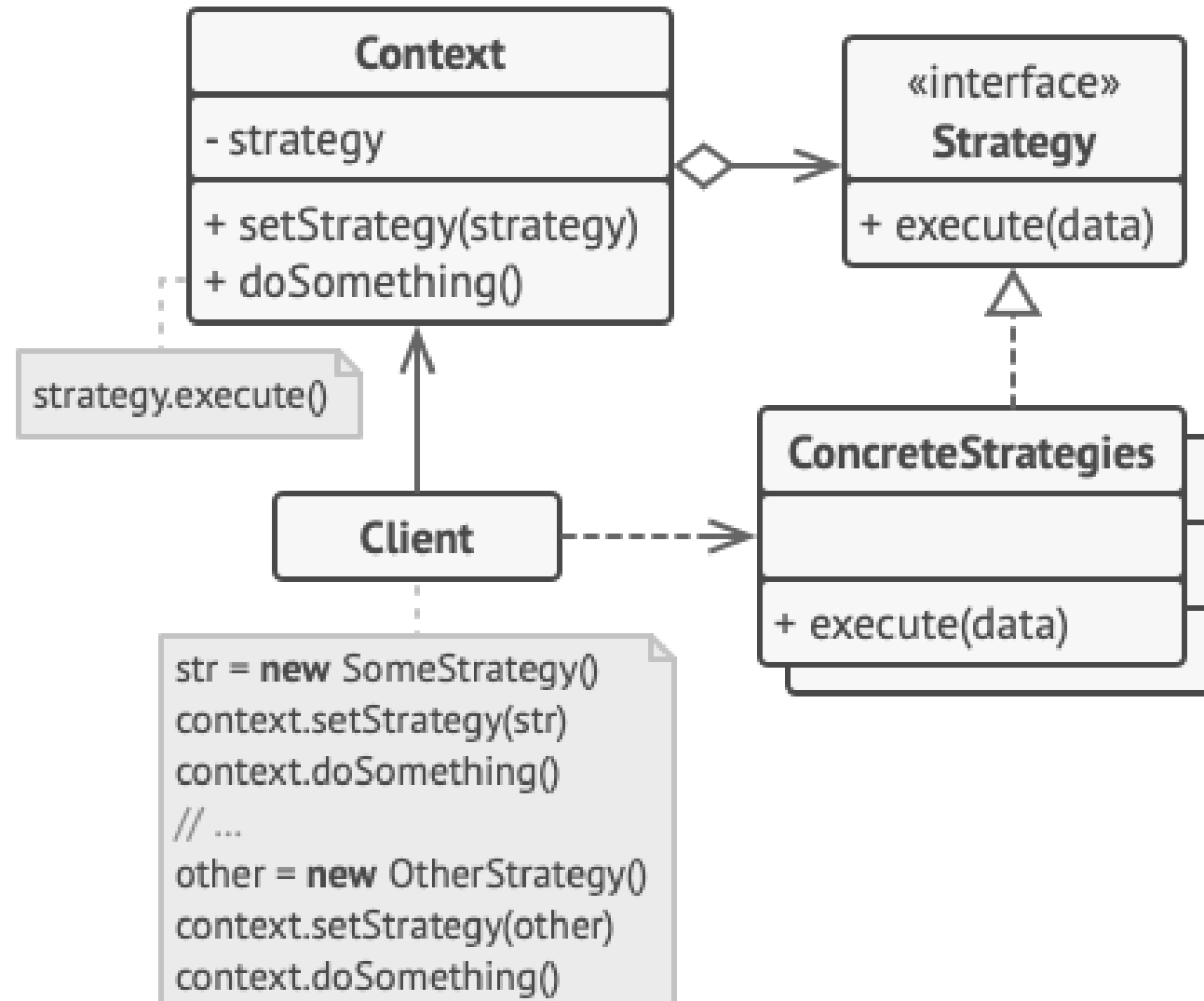
Стратегия (Strategy)



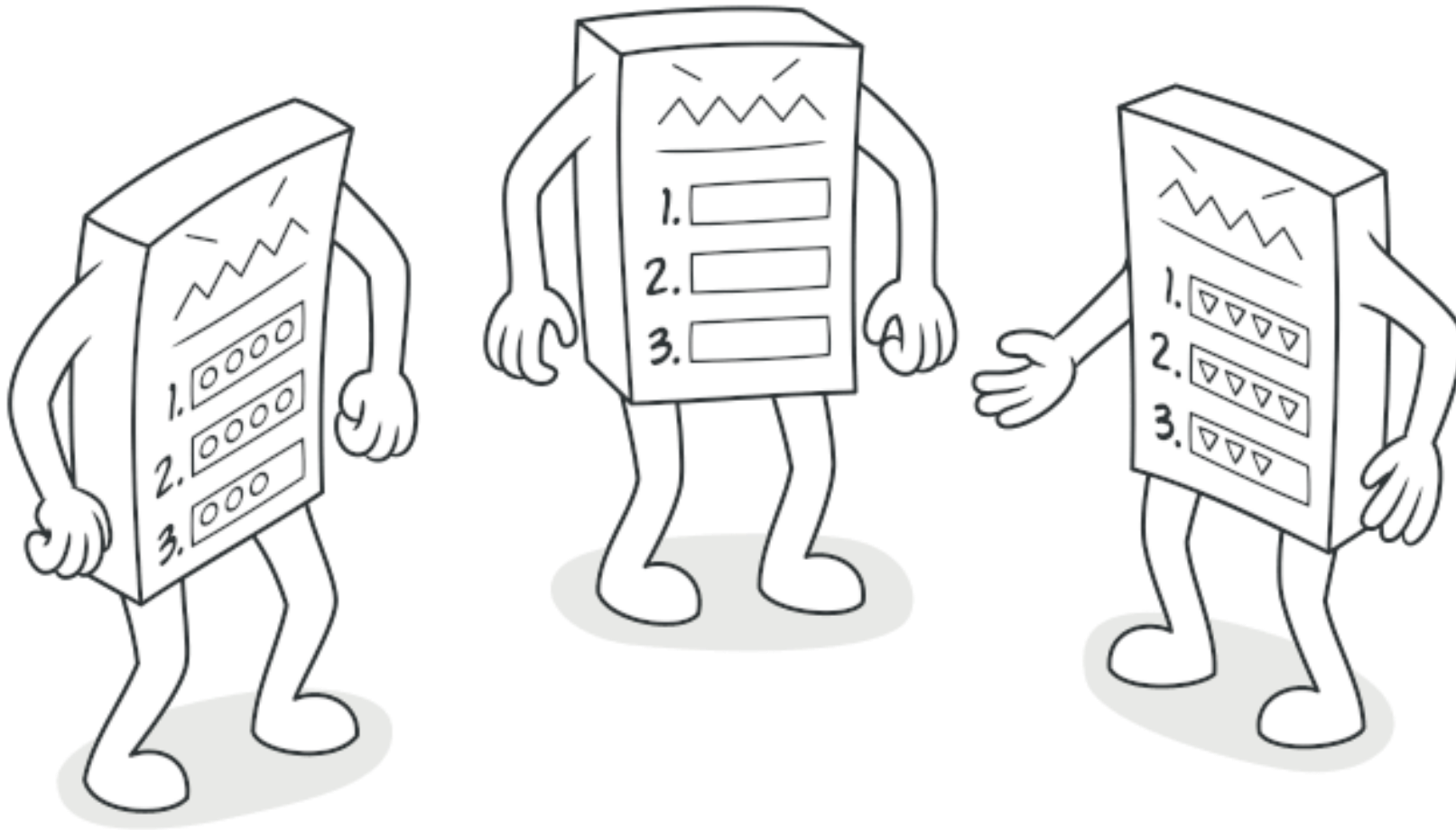
Постановка проблемы



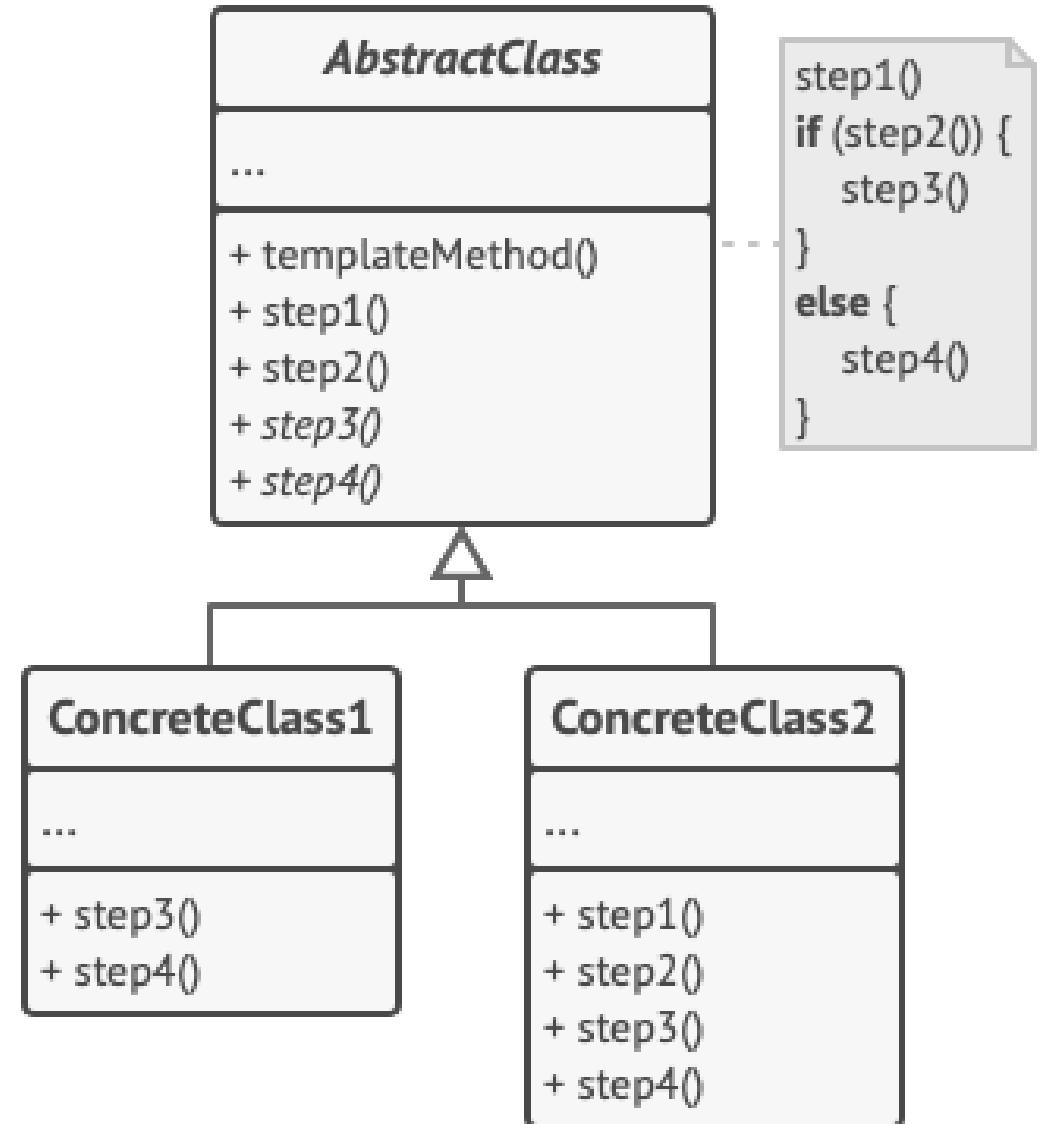
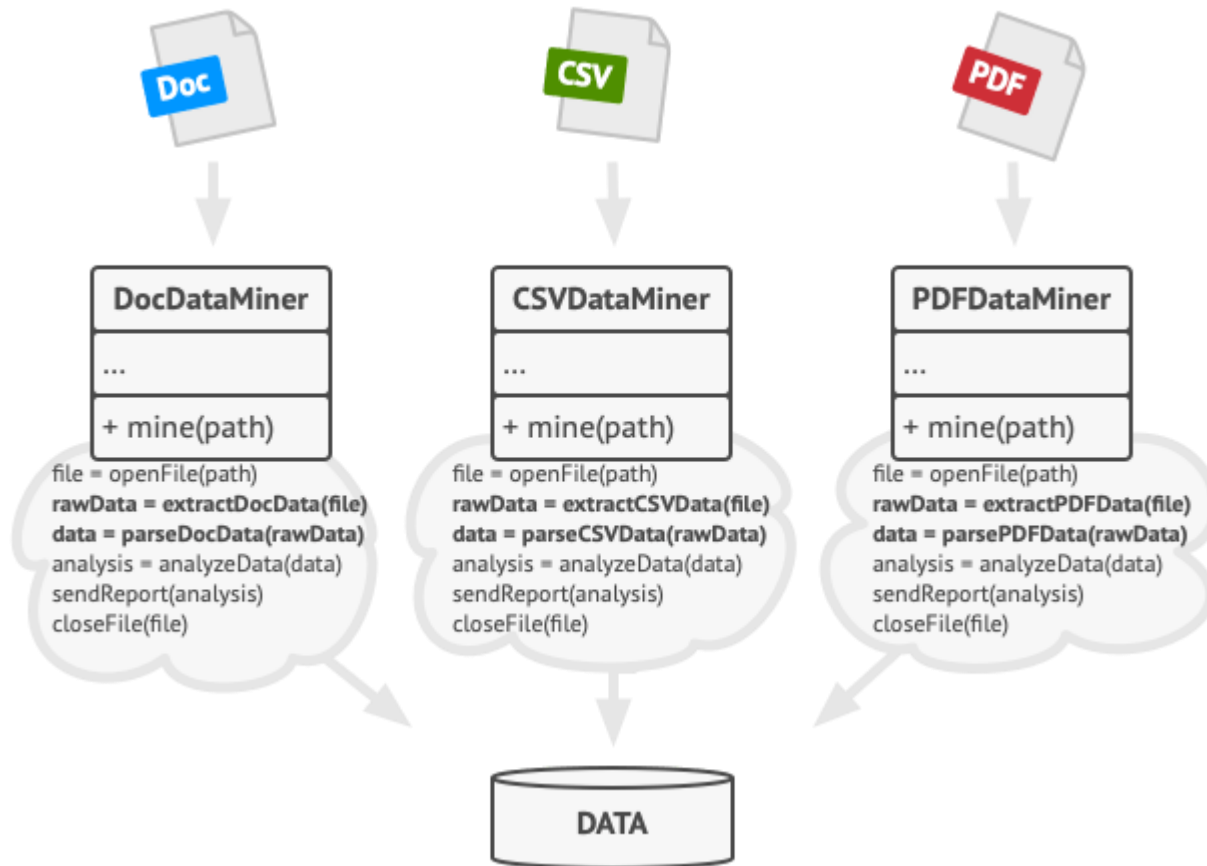
Структура



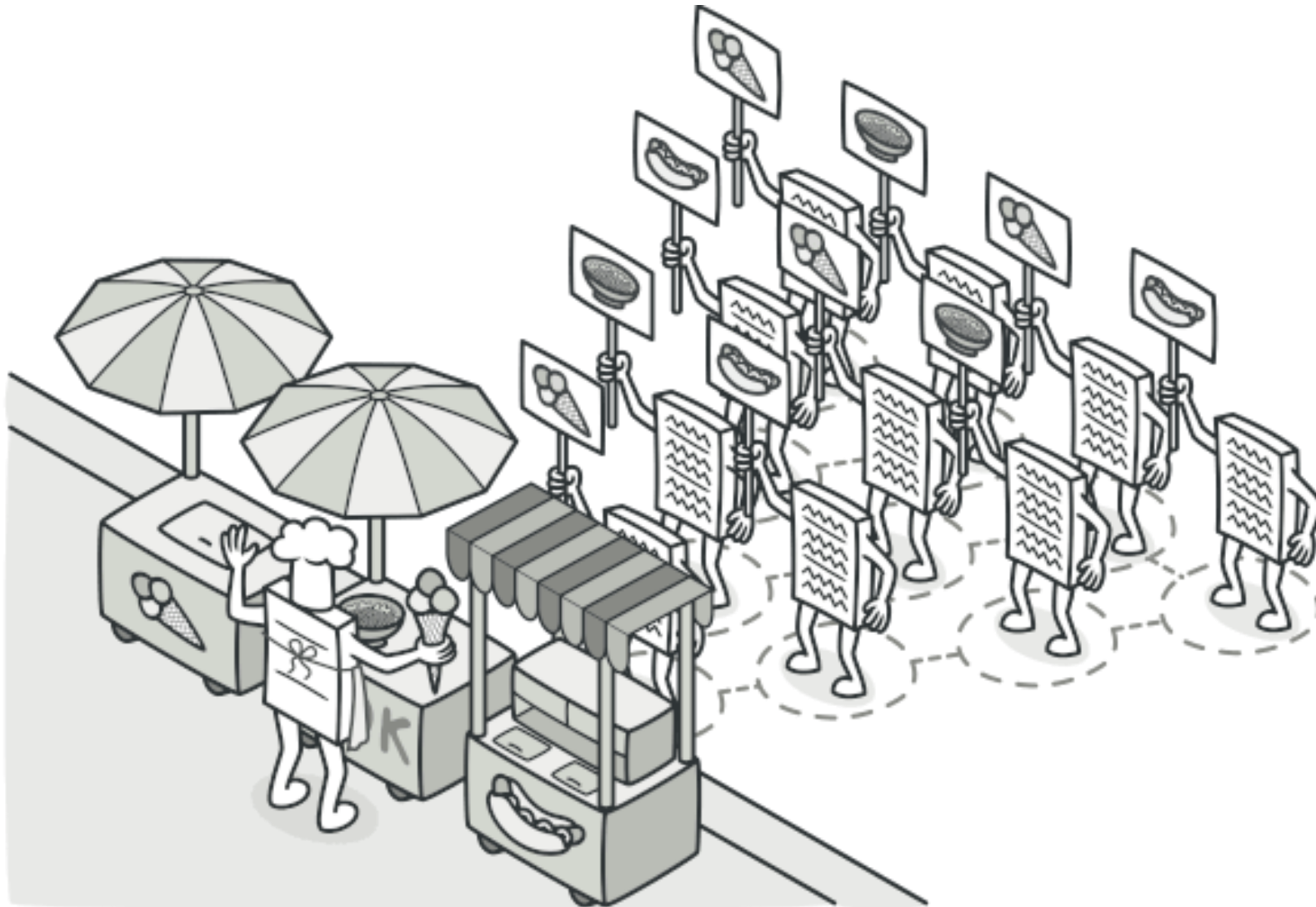
Шаблонный метод (Template Method)



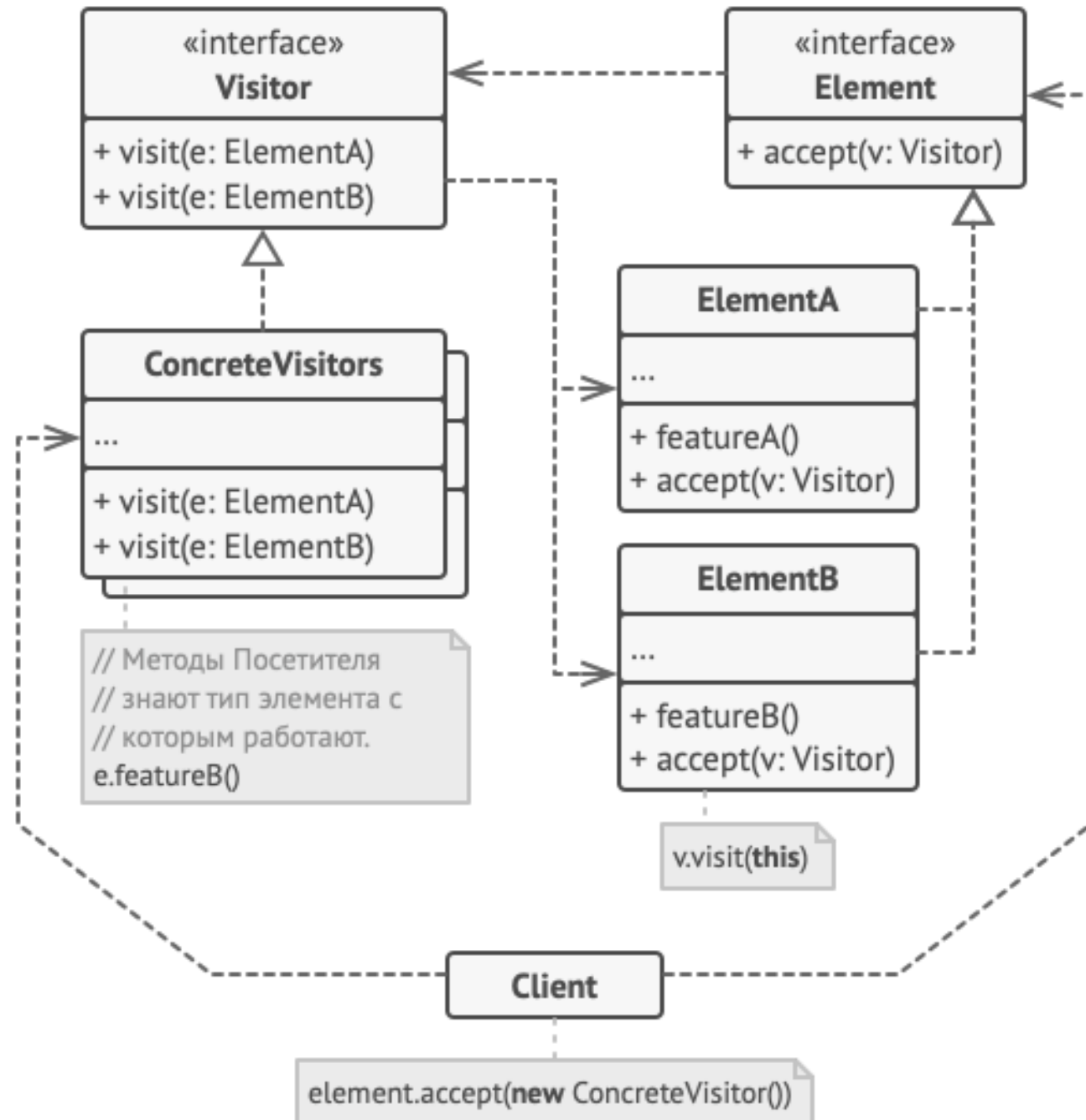
Структура



Посетитель (Visitor)



Структура



Применение паттернов

- ☐ контекст
- ☐ задача
- ☐ решение

Литература

- Приемы объектно-ориентированного проектирования (см. слайд 3)
- Паттерны проектирования (см. слайд 3)
- <https://refactoring.guru/ru/design-patterns>